

HDFS-distribirani fajl sistem

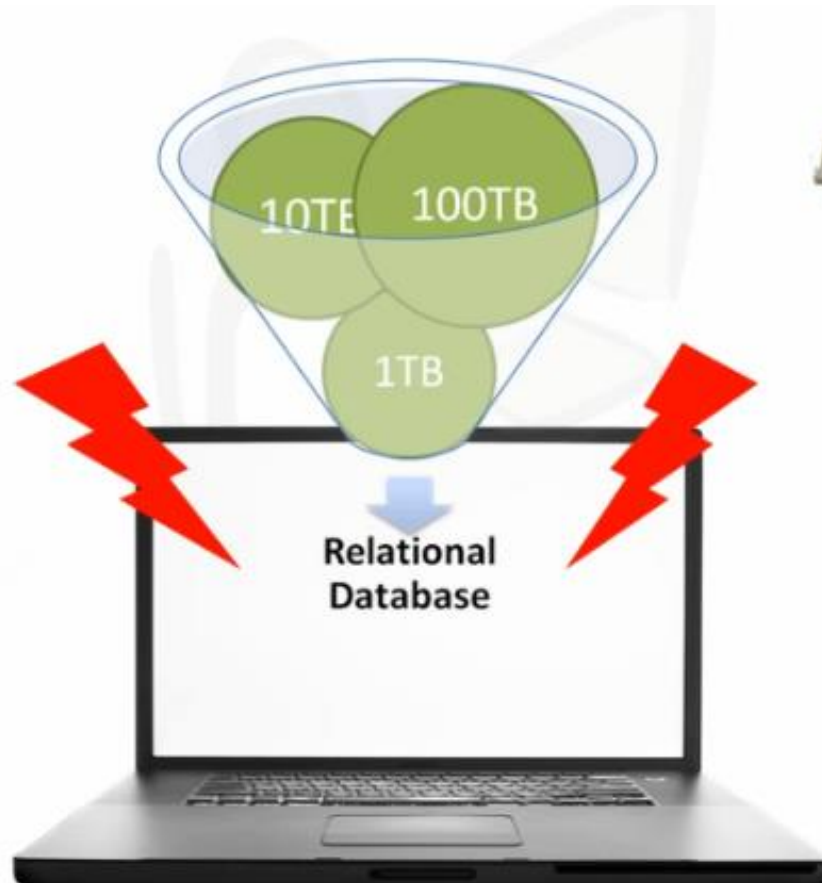
Motivacija

- Pretpostavimo da imamo 1GB podataka koje treba obraditi
 - Podaci su smešteni u relacionoj BP na desktop računaru i računar nema problem da upravlja ovim podacima
- Vaša kompanija počne da raste i podaci koje treba obraditi dostignu 10 GB, a onda 100GB
 - Računar koji obrađuje podatke dostiže svoje granice

Problemi

- Kupovina novog računara rešava problem samo nekoliko meseci, ako podaci počnu da rastu na 10 TB, pa na 100 TB
- Šta više, postoje zahtevi da vaša aplikacija koristi nestrukturane podatke koji potiču iz izvora kao što su Facebook, Twitter, RFID čitači, senzori i sl.
- Vaš menadžment ima zahtev za informacijama i iz relacione baze podataka i iz nestrukturnih podataka i to što pre.

Zahtevi



Motivacija

- Pretpostavimo da je moguće imati disk kapaciteta 500TB.
- Ovo je dovoljno za skladištenje više od 20 milijardi web strana (neka je prosečna veličina strane 20KB)
- U slučaju da samo želite da pregledate tih 500TB biće vam potrebno više od 5 meseci ako disk ima brzinu prenosa podataka sa diska u memoriju od 40 MB/sec.
- Zamislite vreme da potrebno obradu tih podataka!
- Potrebno je drugo rešenje!

Motivacija

- Potreba za više CPU-a: Da bi se programi izvršavali brže
- Potreba za više diskova: Savremene aplikacije zahtevaju rad sa velikom količinom podataka, a sa više diskova I/O se može izvoditi u paraleli

Eksplorzija podataka

- Od 2012. godine svakog dana se kreira oko 2.5 eksabajta (2.5×10^{18}) podataka.
- Do 2020. godine, je proizvedeno 40 zetabajta (40×10^{21}) podataka, 300 puta više od količine koja je postojala pre samo 9 godina.
- Do 2025. se očekuje da bude proizvedeno 160 ZB.
- Odakle dolazi ovoliko podataka?

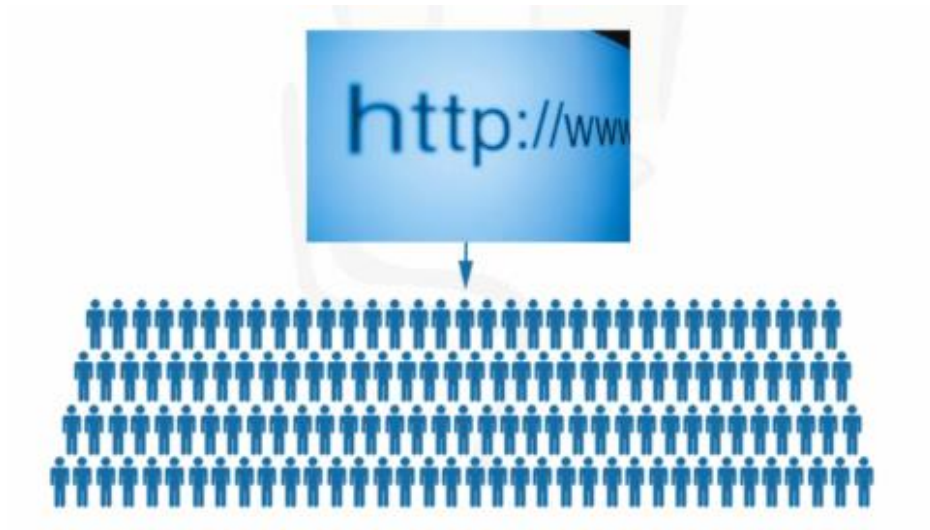
Eksplorzija podataka

- Skupovi podataka se prikupljaju sa jeftinih i mnogobrojnih uređaja koji poseduju određene senzore kao što su: RFID čitači, kamere, mikrofoni, senzori bežičnih mreža i sl. stoga smo svedoci eksplozije prikupljenih podataka širom sveta
- Ovi podaci mogu biti nestrukturirani i da rastu tako brzo da je teško upravljati njima korišćenjem baza podataka



Ljudi kao generatori podataka

- 2 milijarde Internet korisnika



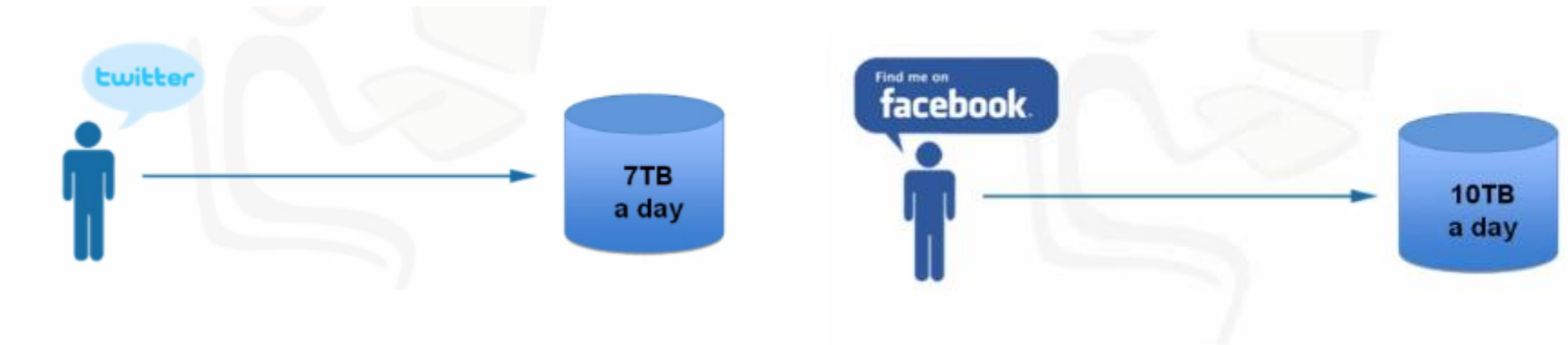
Ljudi kao generatori podataka

- Osnovni razlog se može naći u činjenici da 6 od 7 milijardi ljudi na svetu poseduje mobilni telefon.
- Kako infrastruktura postaje dostupnija i jeftinija, upotreba mobilnih telefona (samim tim i količina podataka koja se generiše) raste eksponencijalno.

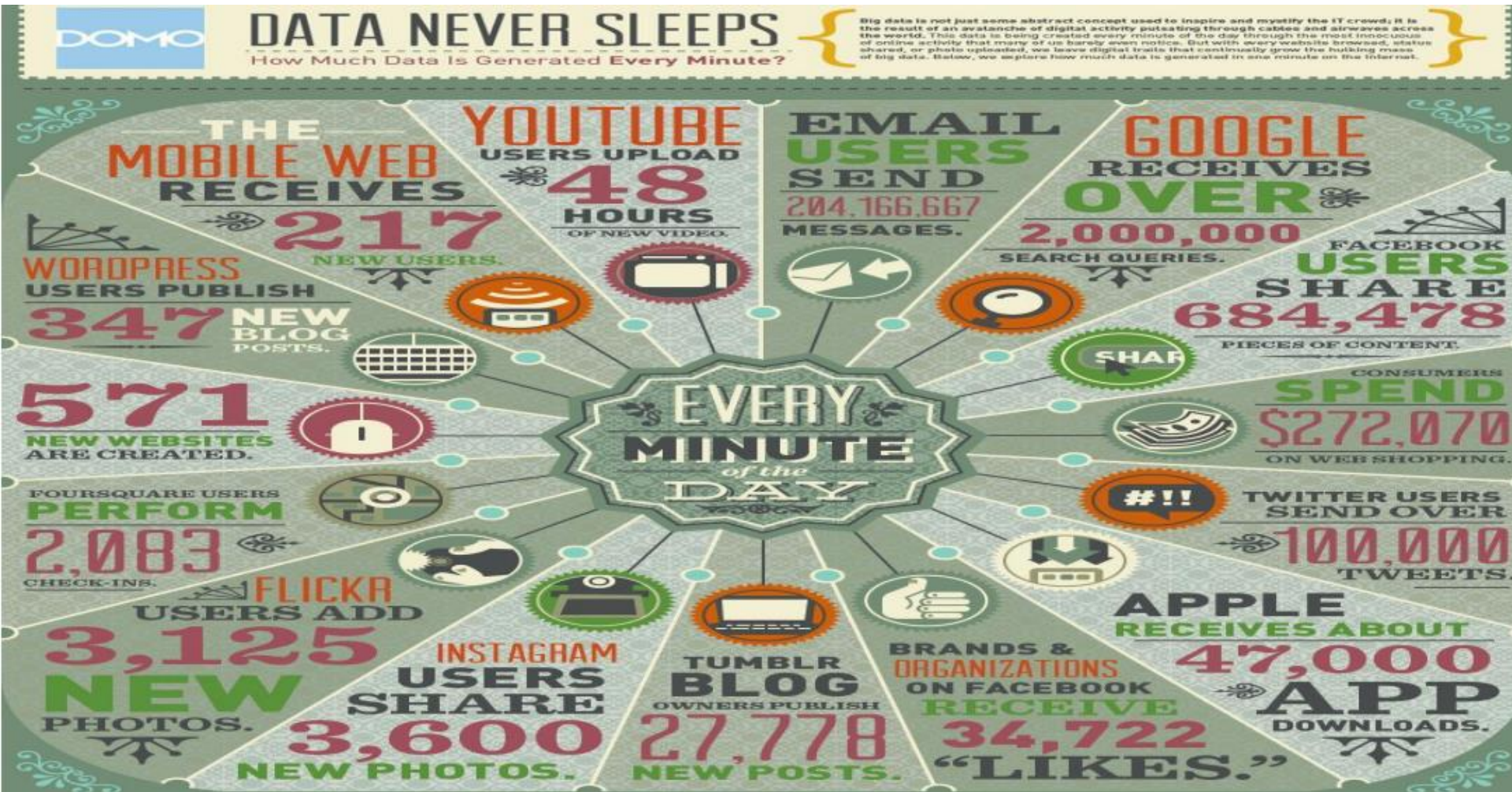


Ljudi kao generatori podataka

- 7 TB podataka generisano na Twitteru svakog dana, 10 TB podataka generisano na FaceBooku svakog dana



Ljudi kao generatori podataka



Ljudi kao generatori podataka

- Zamislite sve podatke koje lično generišete svaki put kada izvršite plaćanje karticom, kada objavite nešto na društvenim mrežama, vozite, posetite lekara i slično.
- Sada zamislite sve te podatke zajedno sa podacima svih ljudi, korporacija i organizacija na svetu.
- Preko zdravstva do društvenih mreža, od biznisa do auto industrije, ljudi generišu više podataka nego ikada ranije.

Ostali generatori podataka

- Veliki Hadron Collider akcelerator u Švajcarskoj proizvede 15 PB podataka za 1 godinu
- Avion Boing 737 generiše oko 240 TB podataka tokom jednog leta sa jednog na drugi kraj Sjedinjenih Američkih Država

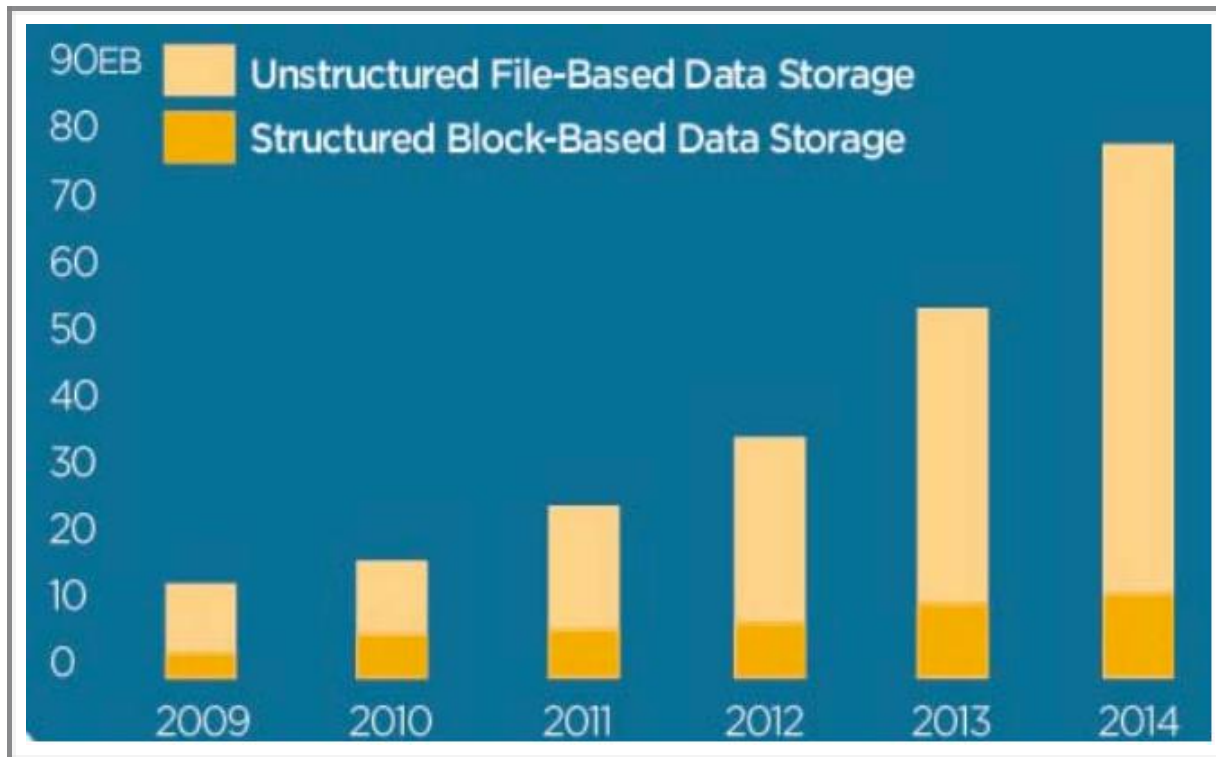
U skoroj budućnosti

- “**IBM Research** and Dutch astronomy agency **Astron** work on new technology to handle **one exabyte of raw data per day** that will be gathered by the world largest radio telescope, the **Square Kilometer Array**, when activated in **2024**”.

Raznolikost podataka

- Podaci su veoma retko u obliku koji je pogodan za obradu. Čest slučaj u *Big Data* sistemima je da su podaci raznoliki, i ne mogu se skladištiti u relacionim bazama podataka.
- Podatke mogu predstavljati tekstovi sa socijalnih mreža, slike, ili *raw(sirovi)* podaci sa senzora.
- Nijedna od navedenih stavki nije u pogodnom obliku za integraciju sa aplikacijom, pa skladištenje podataka prevazilazi tradicionalni pristup gde su podaci smešteni u određeni okvir koji propagira relaciona šema i gde se unapred znaju apsolutno sve veze između podataka.

Raznolikost podataka



Zaključak

- Velika količina podataka je opšti naziv za skupove podataka toliko velike ili složene da se za rukovanje njima ne mogu koristiti tradicionalne aplikacije.
- Izazovi koji se javljaju obuhvataju prikupljanje, skladištenje, pretragu, analizu podataka, prenos podataka ili njihovu vizualizaciju.
- Analize velikih skupova podataka mogu doprineti određivanju trendova u poslovanju, predviđanju i sprečavanju katastrofa, borbi protiv kriminala i slično

Rešenje

- Šta je rešenje?

- Grace Hopper (poznati naučnik u oblasti CS): “In pioneer days they used oxen for heavy pulling, and when one ox couldn’t budge a log, they didn’t try to grow a larger ox. We shouldn’t be trying for bigger computers, but for more systems of computers”.

Scale-Up: put more resources into the system
to make it bigger and more powerful

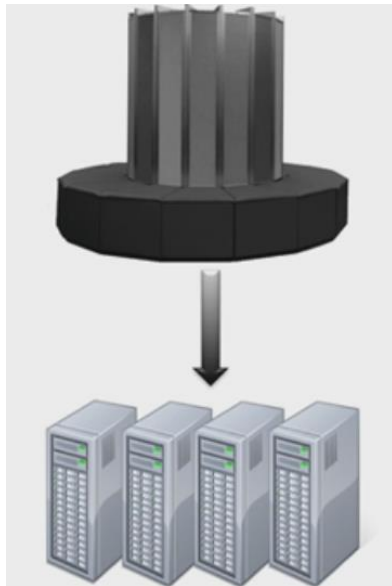


Scale-Out: connect a large number of “ordinary”
machines and create a cluster

Scale-Out is **more powerful** than Scale-Up, and also
less expensive

Rešenje

- Šta je rešenje?
 - Umesto jednog velikog i moćnog računara probati korišćenje sistema koji se sastoji iz više računara (Distribuirani sistemi)



Rešenje-izazovi, MapReduce

- Ovde takođe postoje izazovi:
 - Visok nivo programerske kompleksnosti: organizacija procesa i njihova sinhronizacija
 - Organizacija slanja velike količine podataka sa jednog računara na ostale računare
 - Greške u prisustvu velikog broja računara su neminovnost na koju se mora računati u DS i koja značajno utiče na ispravnost izvršenja aplikacije
- Za sve navedene probleme rešenje se može naći u Hadoop **MapReduce!**

Nastanak Hadoopa

- Eksponencijalni rast podataka je na početku predstavljao izazov velikim kompanijama kao što su Google, Yahoo, Amazon i Microsoft.
- Ove kompanije imaju potrebu da prolaze kroz terabajte i petabajte podataka kako bi saznali koji web sajtovi su popularni, koje knjige se traže na internetu, kao i koje vrste reklama privlače pažnju ljudima.
- Postojeći alati su postali neadekvatni da bi se obrađivali skupovi podataka ovog obima.
- Google je bio prvi koji je publikovao MapReduce – sistem koji oni koriste kako bi obradili podatke tog obima.

Nastanak Hadoopa

- 2003. Google objavljuje svoju klaster arhitekturu i svoj distribuirani fajl sistem (GFS)
- 2004. Google objavljuje svoj MapReduce programski model kao nadogradnju GFS
 - I GFS i MapReduce su pisani na C++ i zatvorenog su koda; sa Python i Java API-jem dostupnim samo Google programerima
- 2006. Apache & Yahoo objavljuju Hadoop i HDFS
 - Open-sorce (otvorenog koda), Java implementacija GFS i GoogleMR sa skupom API dostupnih svima

Nastanak Hadoopa

- Januara 2008. godine, Hadoop na velika vrata ulazi na tržište kao vodeći projekat kompanije Apache i biva korišćen od strane drugih velikih kompanija u razvoju kao što su Last.fm, Facebook i giganta u svetu novinskih agencija kao što je New York Times.
- Aprila 2008. godine, sada već visoko razvijen Hadoop sistem obara tadašnji svetski rekord i postaje najbrži sistem u istoriji za sortiranje terabajta podataka. Sastavljen od 910 čvorova koji čine klaster, Hadoop uspešno obavlja sortiranje terabajta podataka za samo **209** sekundi.
- Novembra iste godine, Google objavljuje da njihov MapReduce sistem istu količinu podataka sortira za neverovatnih **68** sekundi.
- Nadmetanje se nastavlja, pa tako Hadoop ubrzo odgovara izazovu i ovo vreme smanjuje na **62** sekunde.

Nastanak Hadoopa

- Od tada Hadoop sistem za pretraživanje i analiziranje ogromne količine podataka za kratko vreme kreće da sarađuje sa vodećim IT kompanijama poput IBM, Microsoft, Oracle i nastavlja da se širi u okviru svojih ćerki kompanija: Cloudera, Hortonworks i MapR.
- Danas se Hadoop koristi kao platforma opšte namene za skladištenje i analizu velike količine podataka
 - Istraživanje i razvoj se aktivno nastavljaju...

Hadoop

- Hadoop je open-source Apache Foundation projekat koji implementira MapReduce paradigmu
- To je programski okvir napisan u Java programskom jeziku
- Razijen od strane Douga Cuttinga i dobio ime po imenu igračke njegovog sina
- Hadoop koristi Google MapReduce i Google File System tehnologije kao osnovu

Daug Cutting & Hadoop



Hadoop-osobine

- Hadoop je optimizovan da:
 - rukuje velikom količinom podataka koji mogu biti strukturni i nestrukturni
 - uz korišćenje široko dostupnih i relativno jeftinih računara (commodity hardware)
 - sa dobitkom u performansama
 - i ugrađenom visokom pouzdanošću
- Hadoop replicira svoje podatke na više računara tako što, ako jedan računar otkaže, replicirani podaci će biti obrađeni na nekom drugom računaru

Hadoop-osobine

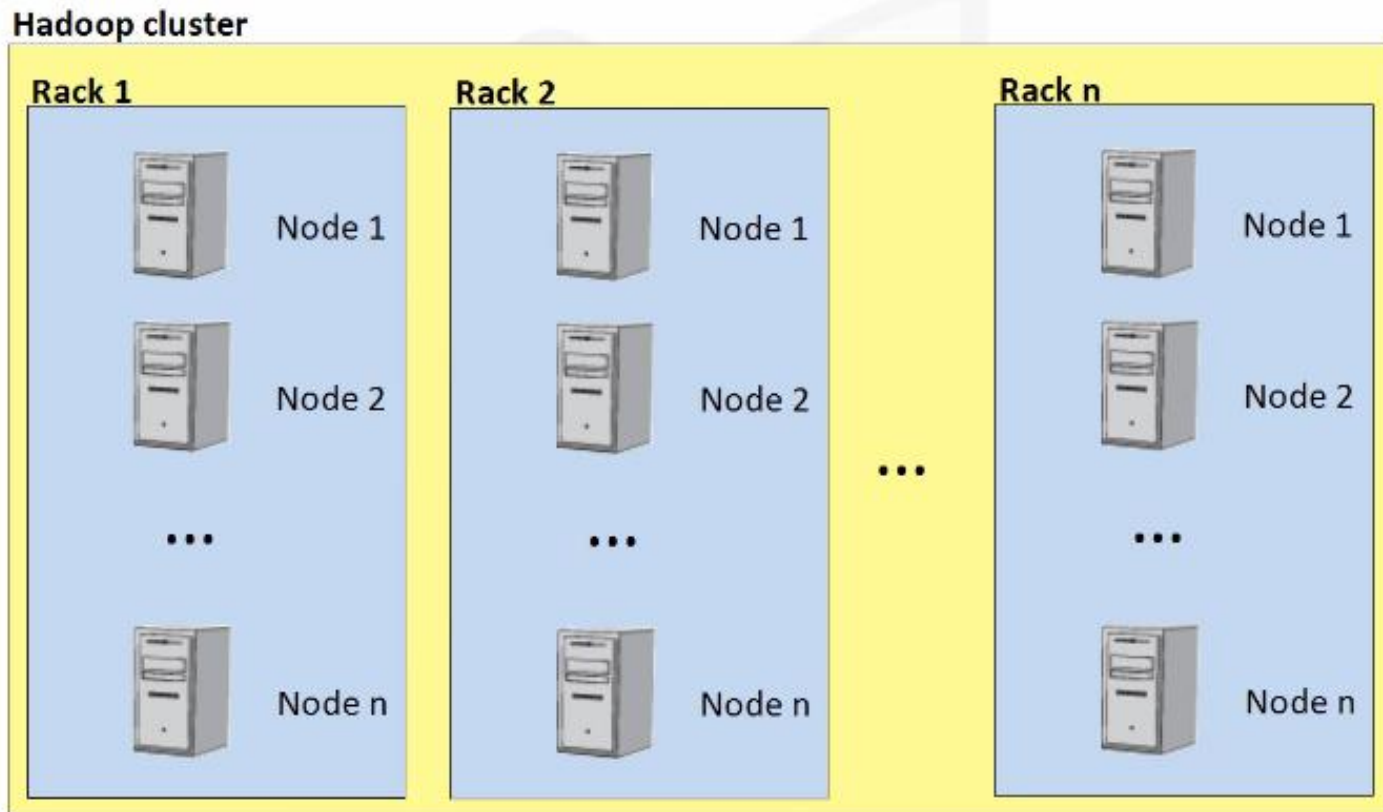
- *Dostupnost* – Hadoop radi na velikoj grupi računara ili u oblaku.
- *Otpornost* – Pošto je Hadoop predviđen da radi na velikom broju računara, on je dizajniran sa pretpostavkama da će često dolaziti do otkaza hardvera i veoma uspešno prevazilazi te probleme.
- *Skalabilnost* – Skaliranje Hadoop klastera se postiže jednostavnim dodavanjem čvorova u klaster.
- *Jednostavnost* – Hadoop omogućava programerima jednostavno pisanje paralelnog koda.

Arhitektura

- Postoje dve glavne komponente Hadoop-a:
 - Distributed File System (DFS) komponenta
 - U slučaju Hadoop ima naziv HDFS i služi za skladištenje podataka na klasteru
 - Map-Reduce komponenta (MR) komponenta
 - Programski okvir za izvođenje izračunavanja nad podacima u HDFS
- Node (Čvor) je računar koji predstavlja commodity (standardni, uobičajen) hw koji sadrži podatke
- Rack (Rek) je kolekcija od 30-40 čvorova koji su fizički blizu jedan drugoga i povezani istim mrežnim switchem.

Klaster

- Hadoop klaster je kolekcija rekova



Distribuirani fajl sistem

- HDFS funkcioniše nad postojećim fajl sistemom na svakom čvoru Hadoop klastera
- Hadoop koristi blokove da sačuva fajl ili delove fajla
- Blokovi su veliki po defaultu 64MB (novije verzije Hadoopa 128 MB)
- Hadoop blok je fajl na fajl sistemu koji se nalazi u osnovi



HDFS blokovi

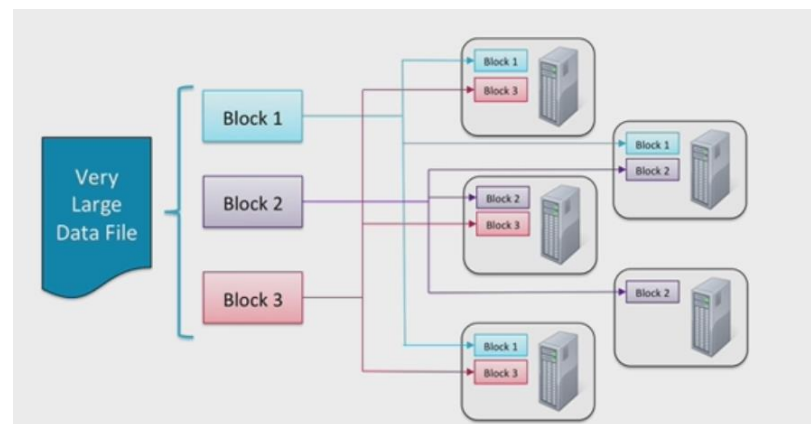


Hadoop distribuirani fajl sistem

- Blokovi imaju sledeće prednosti:
 - Imaju fiksnu veličinu pa je lako izračunati koliko će stati na disk
 - Svojom raspodelom na više čvorova omogućavaju da fajl bude veći nego bilo koji pojedinačni disk u klasteru
 - Blokovi se repliciraju na više čvorova što omogućava HDFS da bude FT (Fault Tolerant)

Hadoop distribuirani fajl sistem

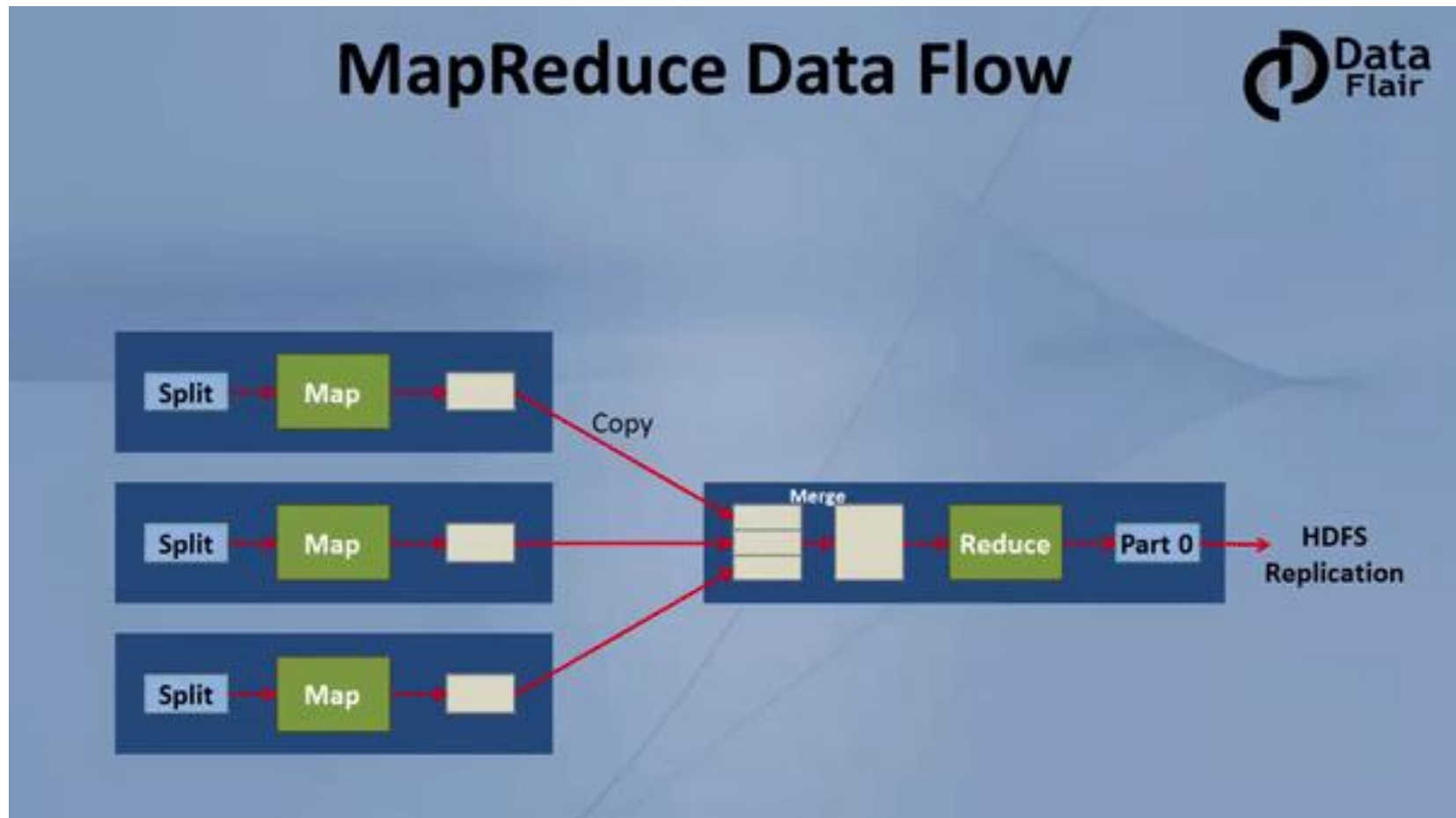
- Ako čvor koji sadrži B1 otkáže, postoje još dva čvora koja sadrže taj blok i omogućavaju da se izračunavanje obavi bez problema
- Faktor replikacije se može promeniti u Hadoop konfiguraciji ili čak postaviti faktor za svaki individualni fajl



MapReduce komponenta

- MapReduce program se sastoji iz dve vrste transformacija koje mogu biti primenjene više puta:
 - Map transformacija
 - Reduce transformacija
- MapReduce job izvršava MapReduce program koji je podeljen na Map taskove koji se izvršavaju paralelno i Reduce taskove koji se izvršavaju paralelno jedni sa drugima

MapReduce komponenta



Hadoop čvorovi

- Postoje tri vrste čvorova u klasteru u odnosu na uloge koje imaju: klijent čvor, *master* (gospodar) čvor i *slave* (sluga) čvor.
- Uloga Klijent čvora je da loaduje podatke u klaster, dostavlja MapReduce posao (job) koji opisuje kako treba obraditi podatke i dobije ili pogleda rezultate obavljenog posla po njegovom okončanju
- Na master čvoru se vrši nadgledanje i rukovodjenje dvema ključnim komponentama Hadoop-a: skladištenje velikog broja podataka (HDFS) i izvršavanje paralelnih izračunavanja nad svim tim podacima (MR).
- Slave čvorovi su čvorovi koji obavljaju skladištenje podataka i izvršavaju izračunavanja.

Hadoop demoni

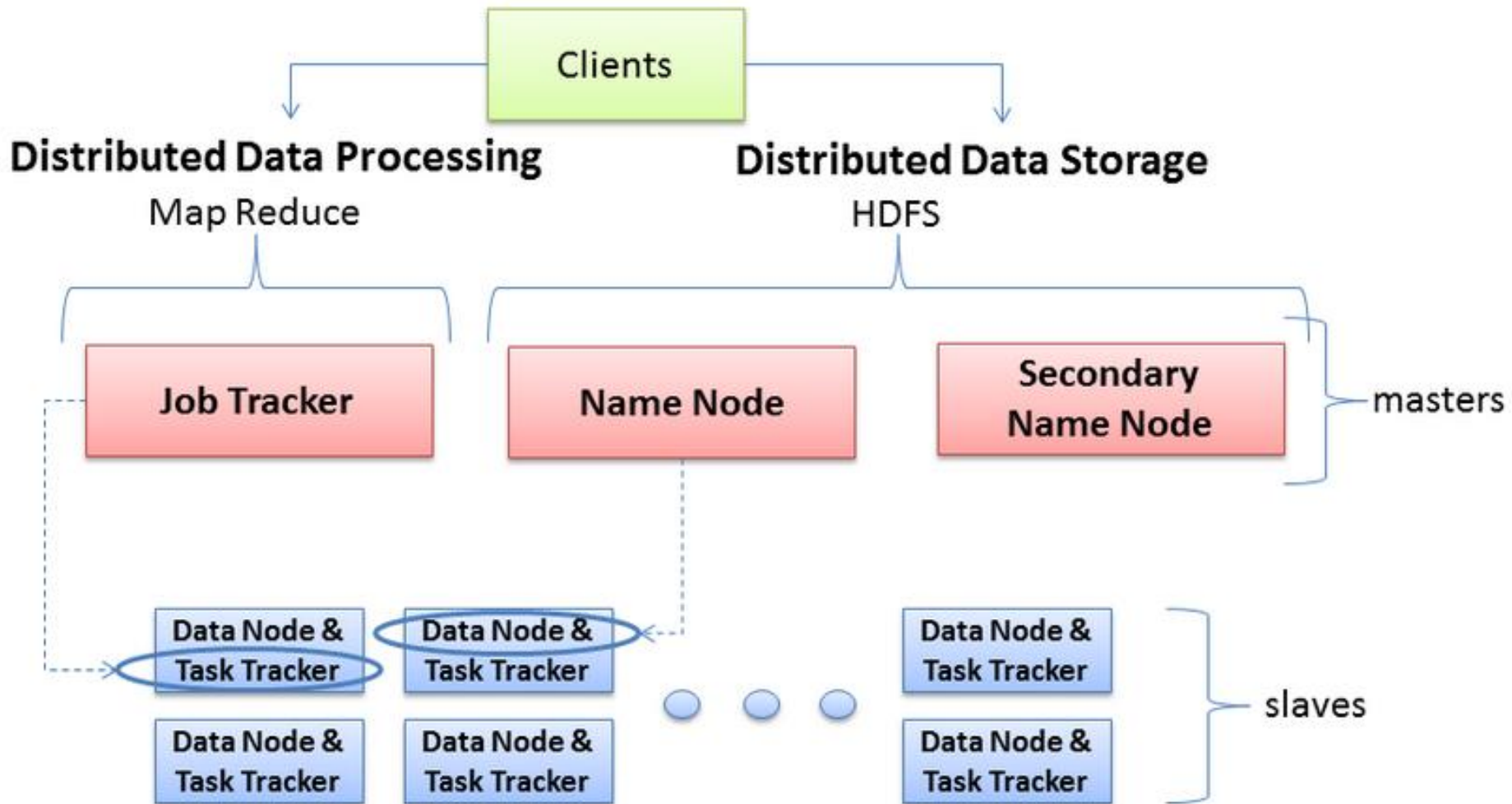
- Prilikom startovanja Hadoopa na čvorovima klastera se startuje skup demona (eng. deamon) koji ima specifične uloge.
- Postoje dve vrste demona Hadoop-a:
 - HDFS demoni
 - MapReduce demoni

HDFS demoni

- U HDFS demone spadaju jedan NameNode (NN) demon i više DataNode (DN) demona
- Postoji samo jedan NameNode i više DataNode-ova u klasteru
- DataNode odgovoran za čuvanje podataka u blokovima, primanje naredbi od NN i davanje informacija NN-u.

HDFS demoni

- Namenode daemon je demon koji se izvršava na glavnom ili master čvoru
- NameNode održava fajl sistem namespace (prostor imena) i svaka promena njegovih karakteristika je zapamćena od strane NameNode.
- NameNode čuva informacije o tome kako su fajlovi podeljeni na blokove, koji *slave* čvorovi čuvaju koje blokove i sam učestvuje u preslikavanju blokova u čvorove
- NameNode čuva informaciju i o vrednosti faktora replikacije



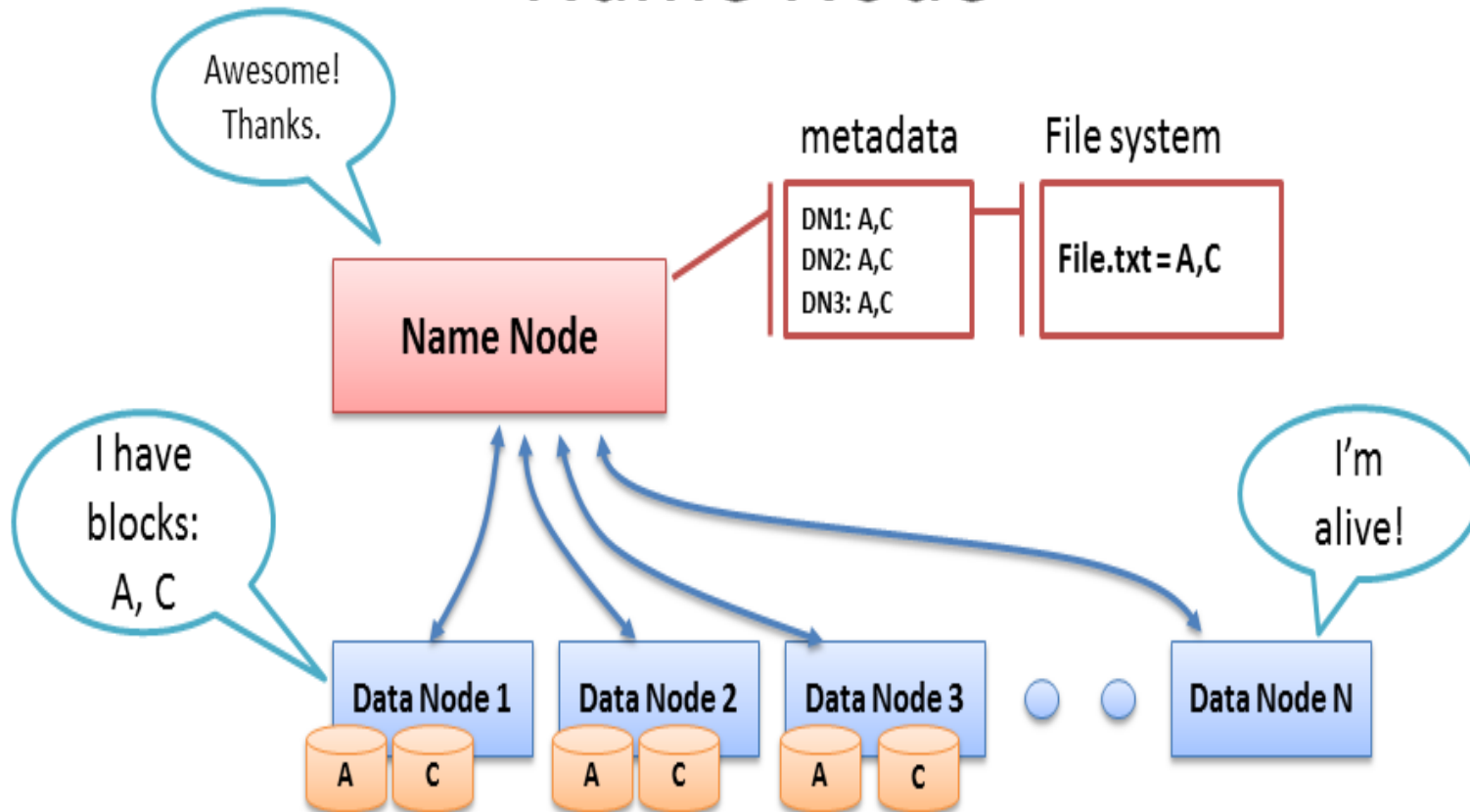
NameNode

- NameNode je centralni kontroler HDFS-a
- NN čuva metapodatke fajl sistema, nadgleda ponašanje DN-a i koordiniše pristup podacima.
- NN ne čuva podatke nad kojima se vrši obrada , on samo ima informaciju o blokovima koji čine fajl i gde su ti blokovi locirani u klasteru
- Takođe vodi računa da svaki blok zadovolji definisani faktor replikacije

NameNode

- Namenode je u direktnoj vezi sa ostalim čvorovima, i za komunikaciju sa DataNode demonima koristi paket određene strukture koji se naziva *heartbeat*
- Ovaj paket nosi takav simboličan naziv jer preko njega Namenode proverava da li je Datanode aktivan, i pored ovoga, ovaj paket prenosi identifikatore blokova podataka koji se nalaze na hostu Datanode demona.
- DataNodovi šalju *heartbeats* svake tri sekunde i svaki deseti heartbeat je izveštaj o blokovima, gde DN obaveštava NN koje blokove čuva
- Ovi izveštaji služe NN da osiguraju da odgovarajući broj replika blokova (najčešće 3) postoji na različitim čvorovima, u različitim rekovima

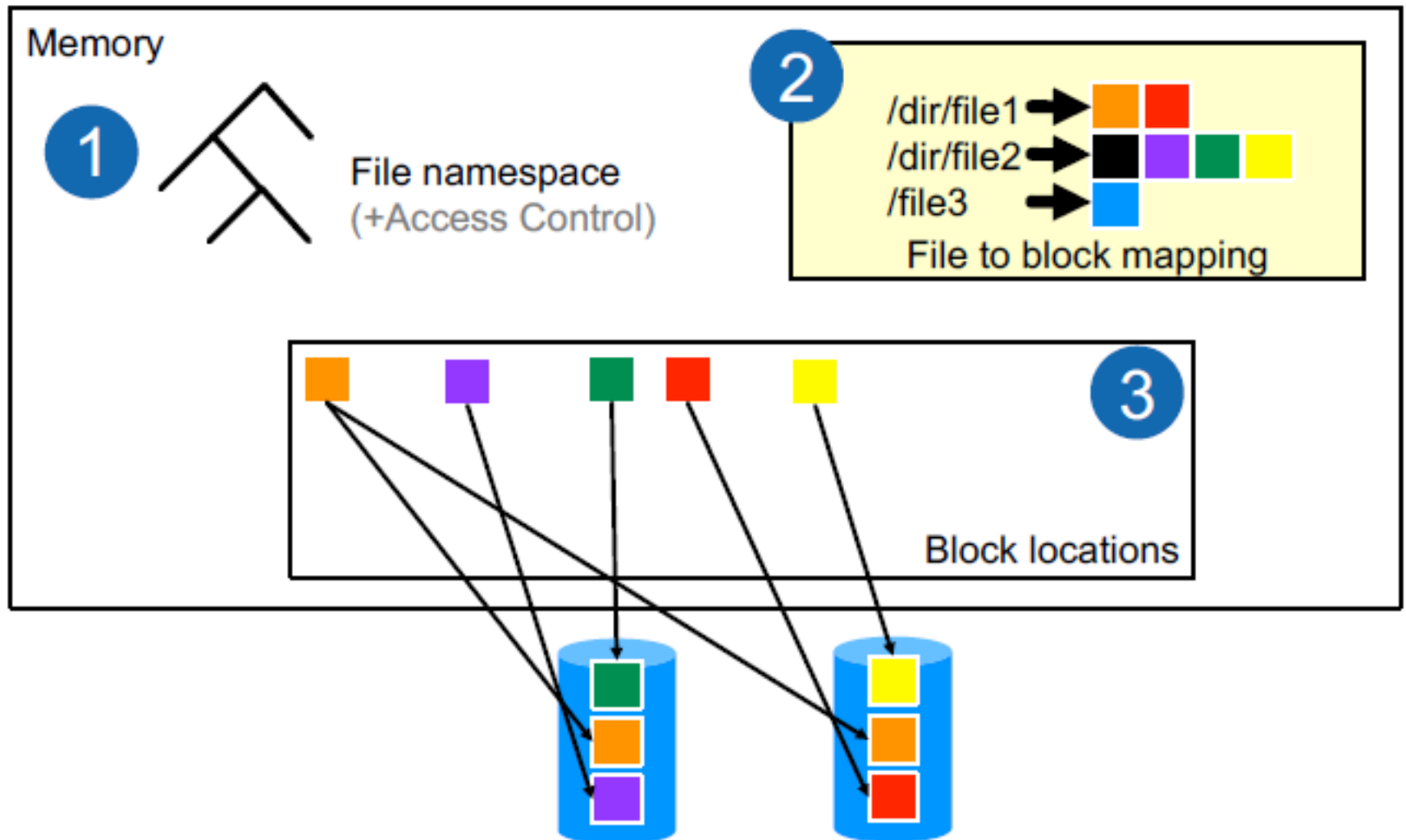
Name Node



NameNode

- NN je kritična komponenta HDFS-a
- Bez njega klijent ne bi mogao da upisuje i čita fajlove iz HDFS-a i bilo bi nemoguće upravljati i izvršavati MR poslove
- Iz tog razloga dobro je da NN bude neki jači server ali i bitno je obezbediti i dodatnu podršku u slučaju problema u radu NN
- Namenode čuva ove podatke u glavnoj memoriji, zato što se ovi podaci u velikim sistemima često menjaju.
- Ovakav pristup je omogućio Hadoop sistemima jednostavno proširenje klastera i nesmetan rad pri određenim tipovima otkaza.

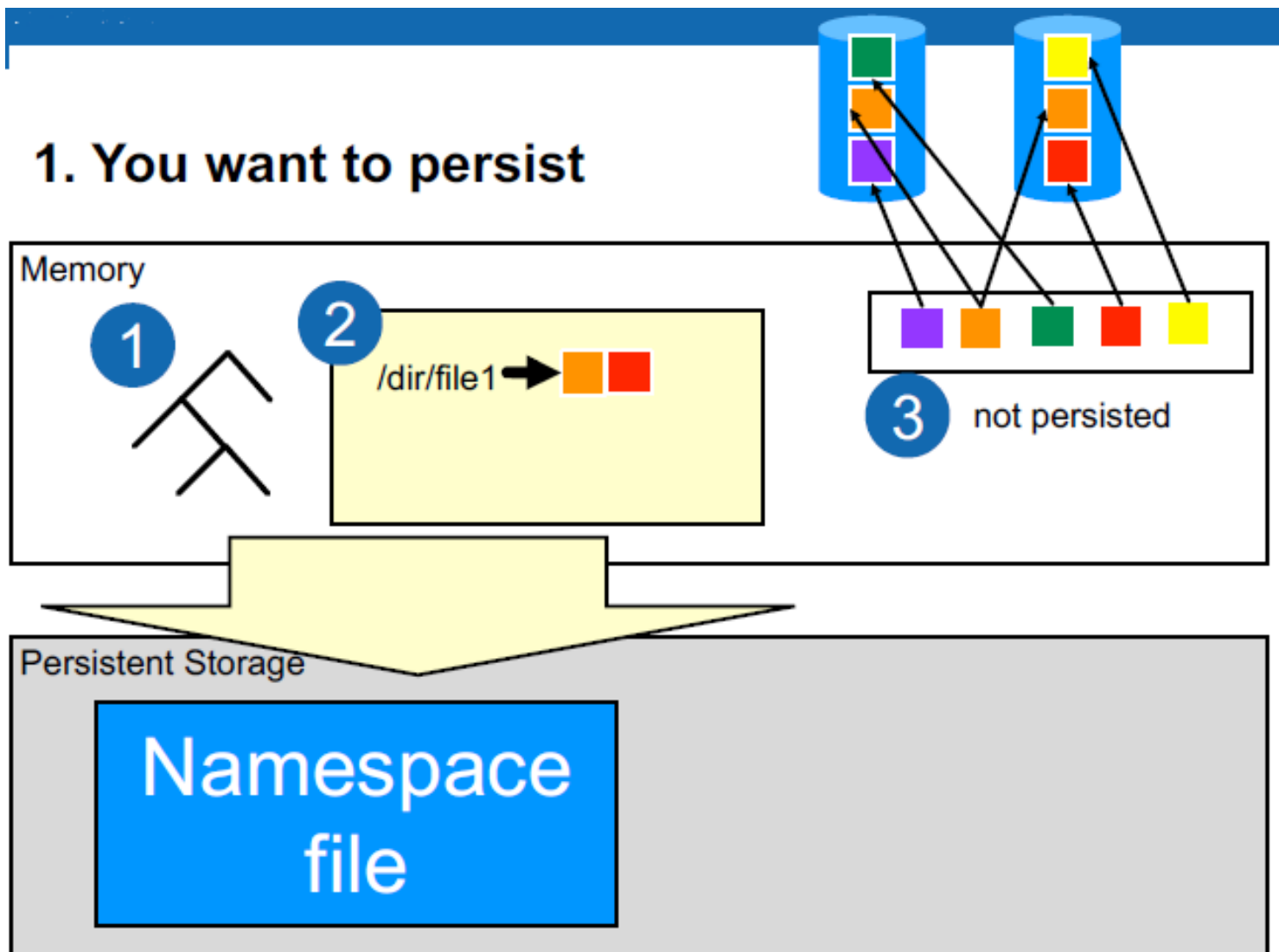
NN glavna memorija



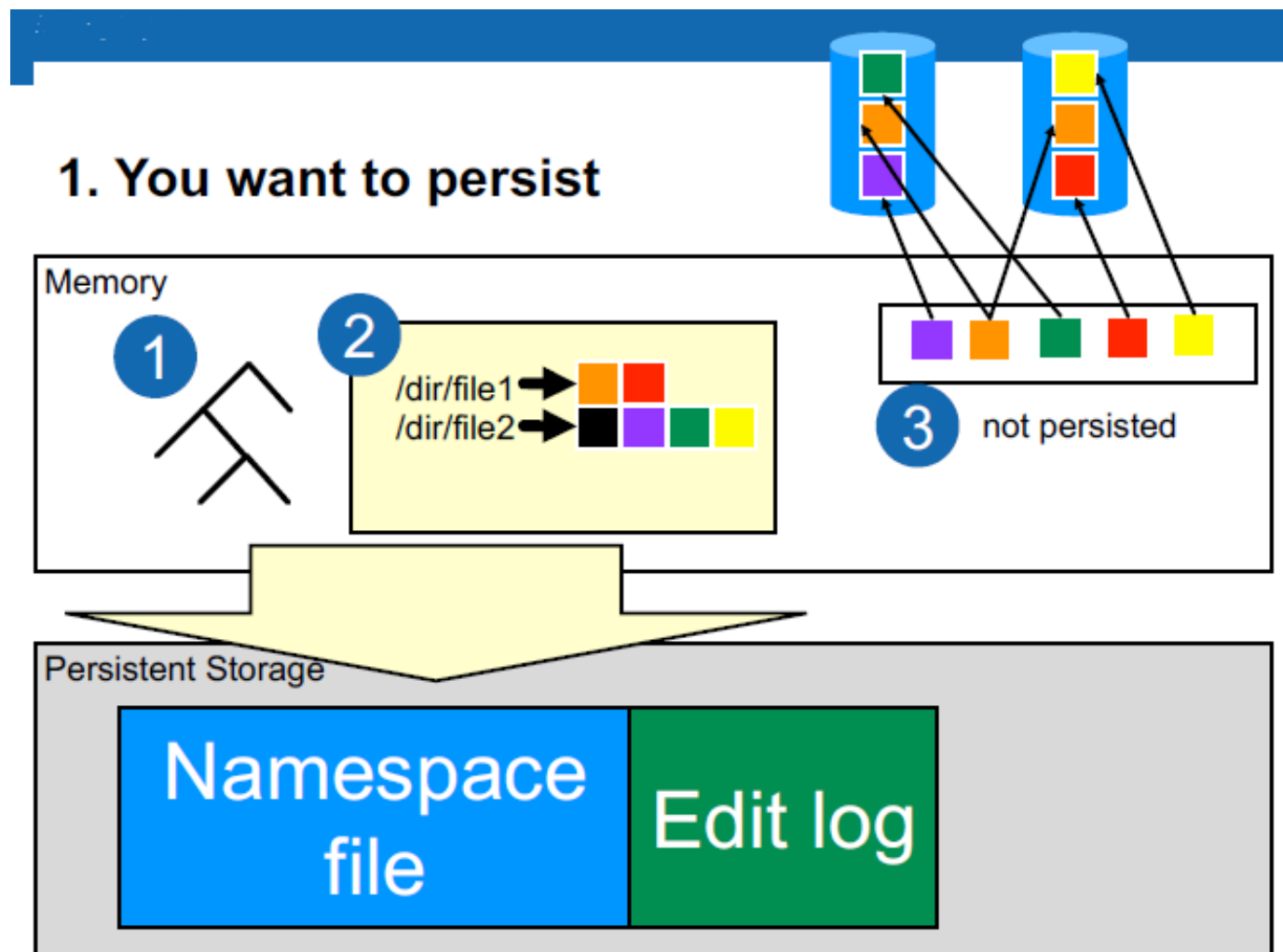
Trajno čuvanje metapodataka NN-a

- Trajno čuvanje metapodataka NN-a na NN lokalnom file sistemu se naziva Checkpoint
- Fajl *fsimage* (ili *Checkpoint image* kod GFS) je fajl u kome se čuva snapshot metapodataka fajl sistema (Namespace file)
- Fajl na lokalnom fajl sistemu NN-a u kome se trajno čuvaju ***promene*** koje se dešavaju u HDFS-u se zove ***edit log***

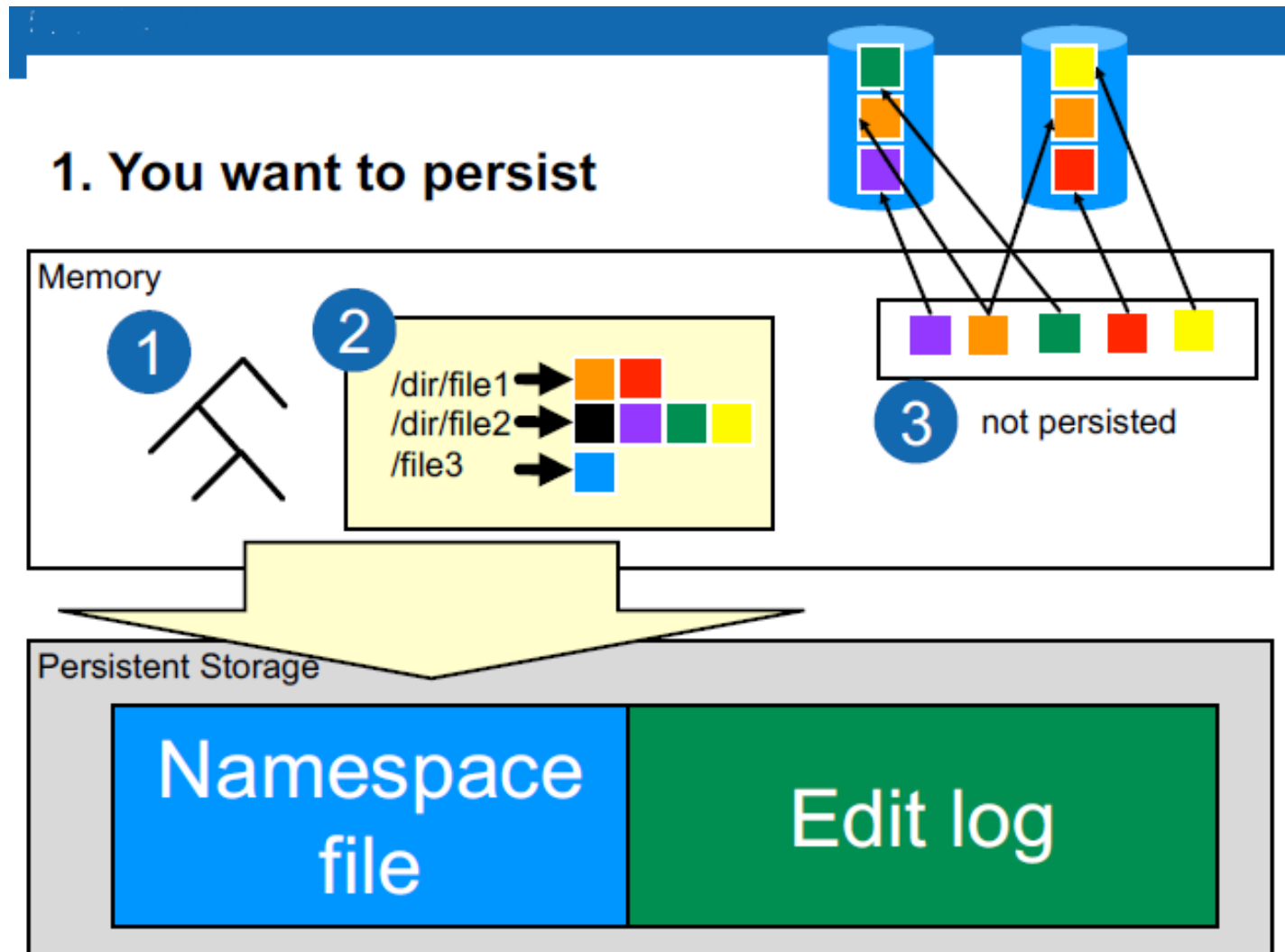
Trajno čuvanje metapodataka NN-a



Trajno čuvanje metapodataka NN-a



Trajno čuvanje metapodataka NN-a

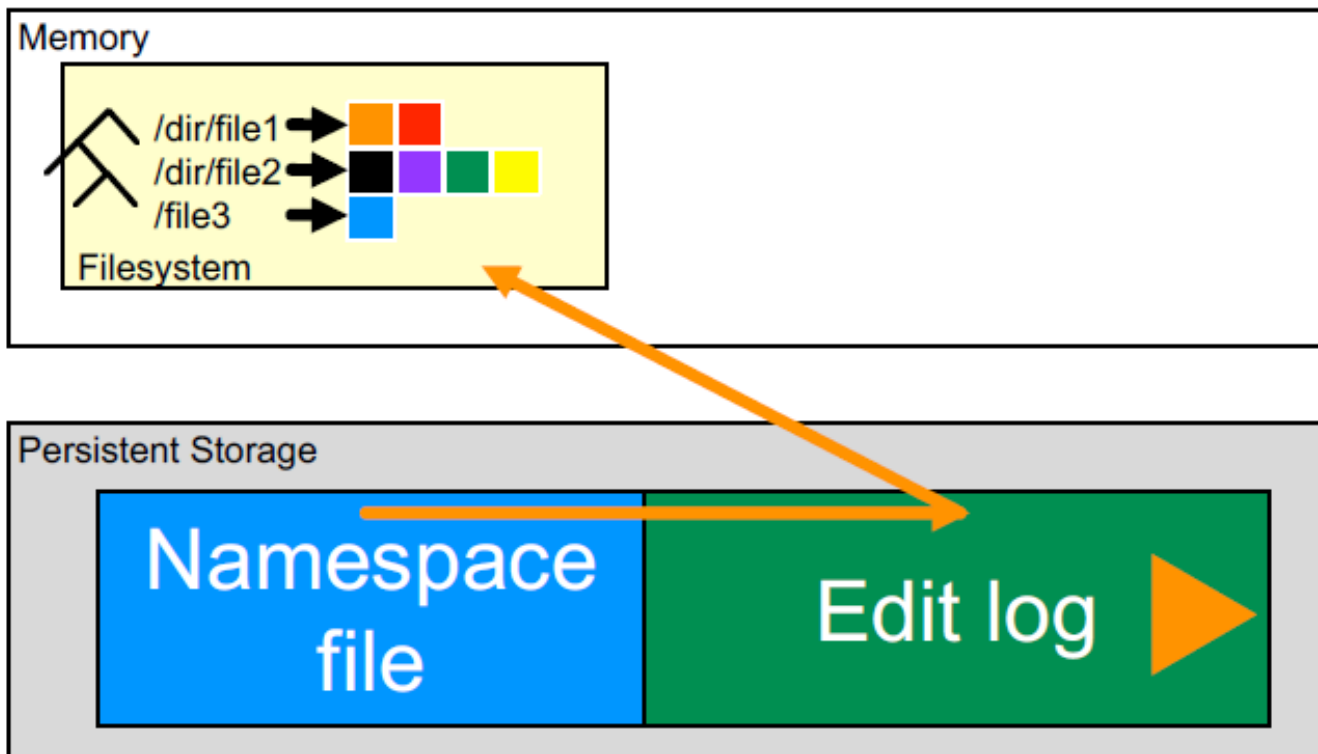


NN Restart

Samo u slučaju kada se restartuje NN, promene iz edit loga se primenjuju na poslednju verziju Namespace fajla(fajl fsImage) da bi se dobila nova verzija metapodataka fajl sistema.

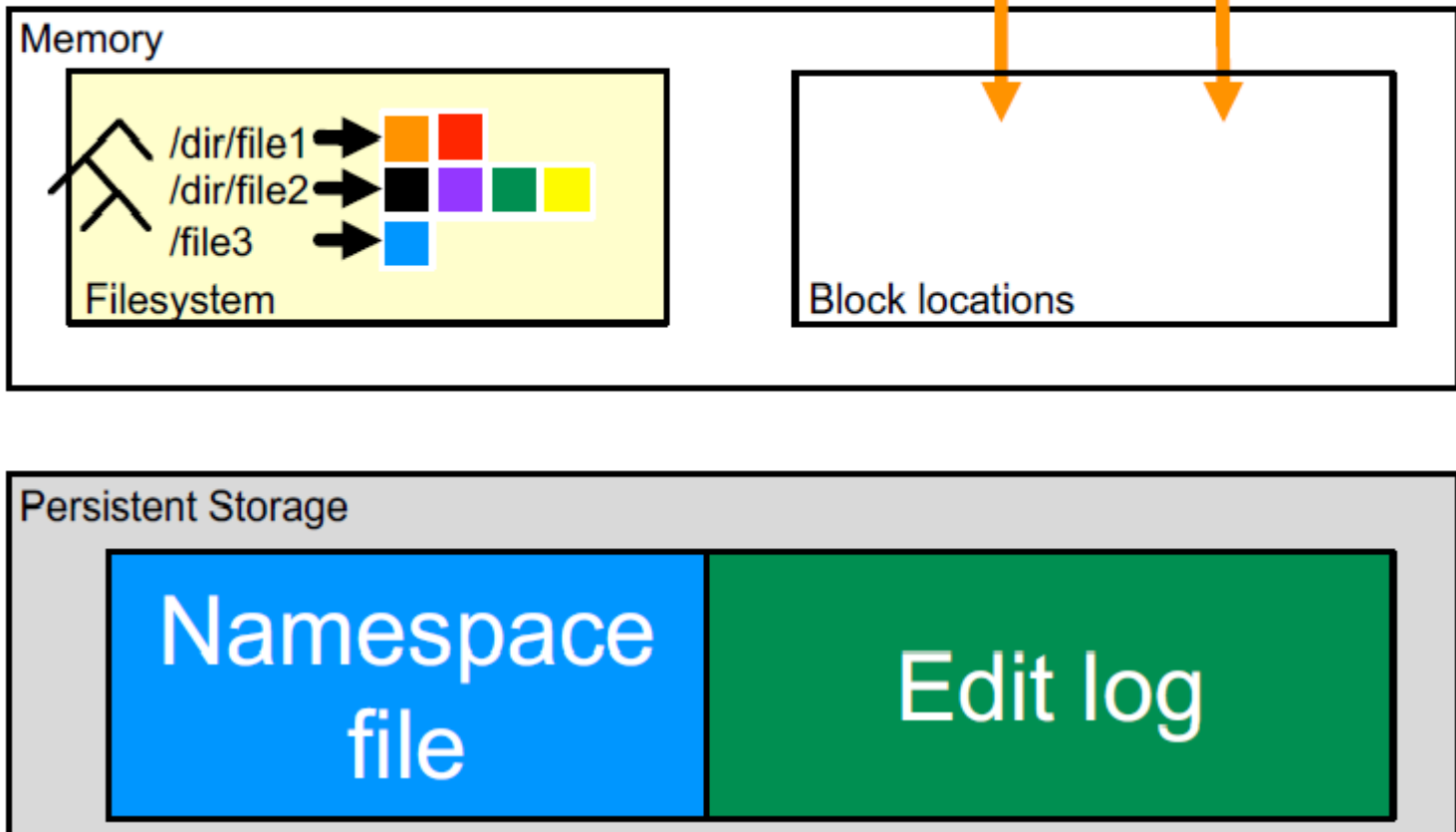
Pošto se u klasterima retko vrši restart, edit logovi mogu narasti mnogo, tako da vreme za restart postaje mnogo dugo

Namenodes: Startup



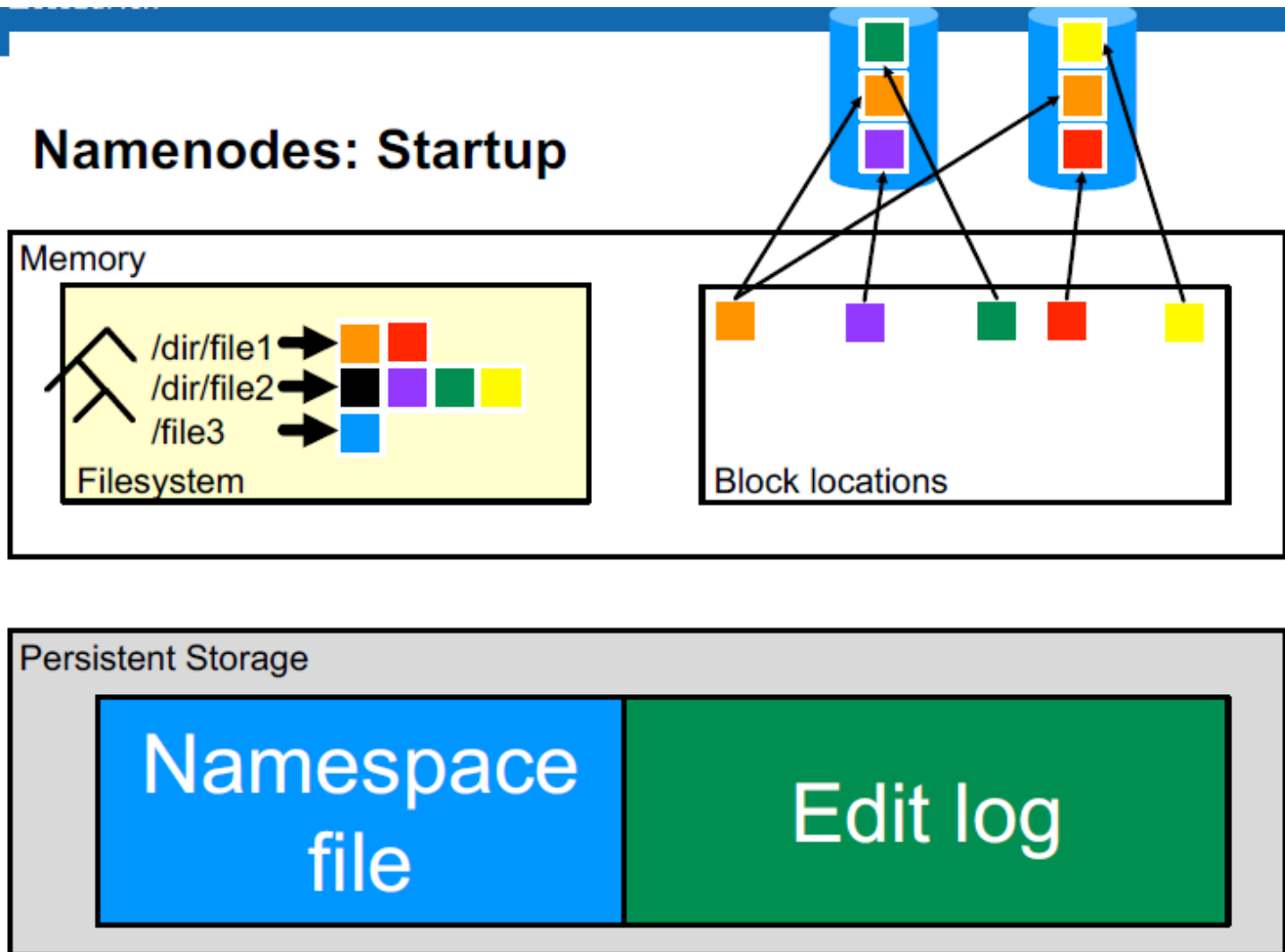
NN Restart

Namenodes: Startup



NN Restart

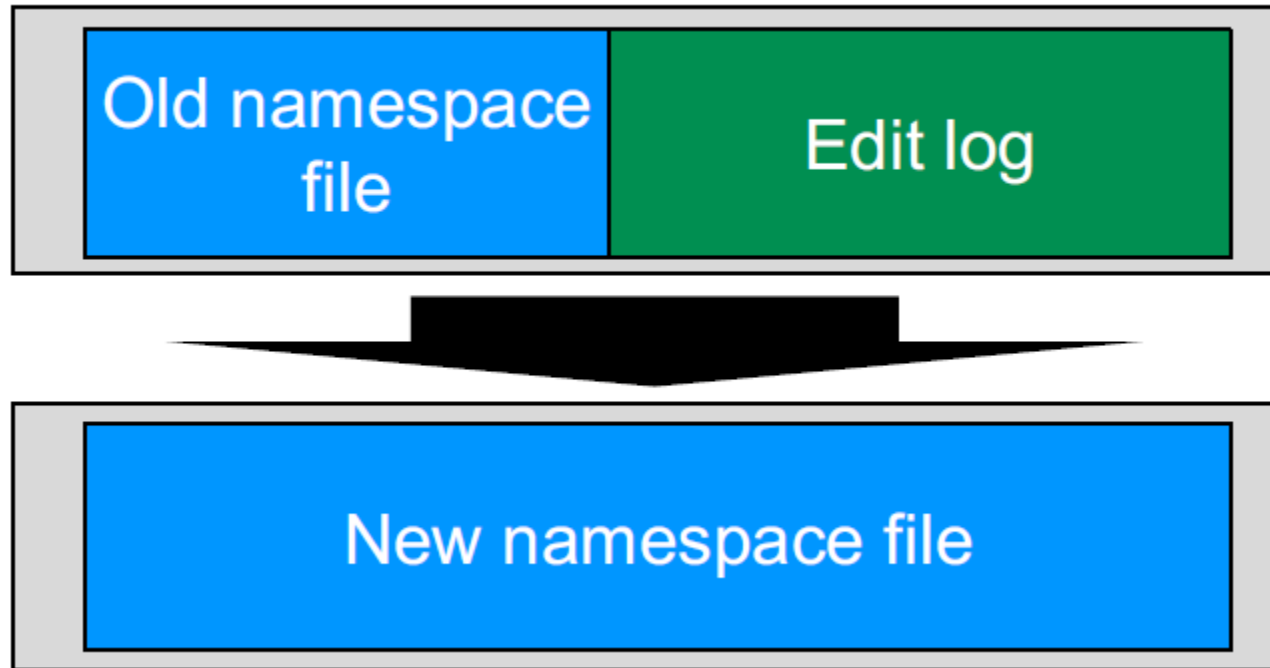
Namenodes: Startup



Sekundarni NN

- CP treba formirati što češće jer veliki edit log povećava vreme za restart NN (potrebni su sati za restart u slučaju edit log-a koji je formiran cele nedelje) a i u slučaju otkaza NN je bitno
- Da bi se prevazišao taj problem koristi se Secondary NN
- Secondary NN je startovan na serveru na kojem nije NN
- Secondary NN radi konkurentno sa NN kao pomoćni demon i on ne predstavlja NN backup

Chackpoint na sekundarnom Name Node



- Secondary NN u regularnim intervalima preuzima edit log od NN, spaja ih lokalno (primenjuje promene iz edit loga na stari CP) i formira novi CP koji vraća NN.
- NN koristi ovaj CP prilikom sledećeg restarta

Rack awareness

- Kao što je već rečeno, svaki blok podataka koji treba da se upiše biće repliciran na više mašina da bi se sprečilo da otkaz jedne mašine dovede do gubitka svih kopija podatka
- Šta bi se desilo kada bi sve kopije podataka bile locirane na mašinama u istom reku koji otkaže?
- Da bi se to izbeglo, potrebno je da se zna gde su DataNodovi locirani u mreži i tu informaciju iskoristiti u odlučivanju gde treba da budu ostale replike. To informaciju ima NameNode!

Rack awareness



Replica 1: same node as client (or random), rack A

Replica 2: a node in a different rack B

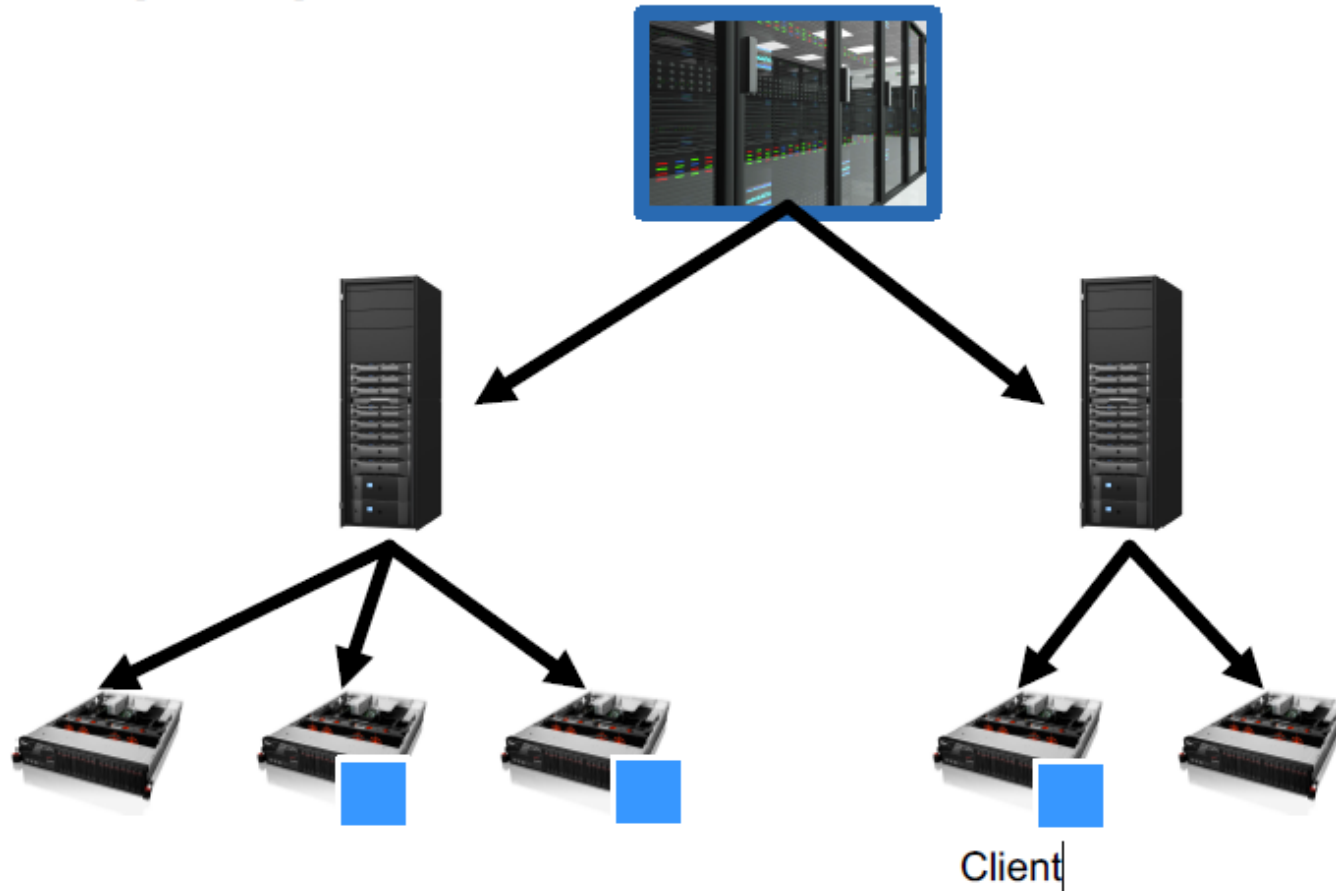
Replica 3: a node in same rack B

Replica 4 and beyond: random, but if possible:

- at most one replica per node
- at most two replicas per rack

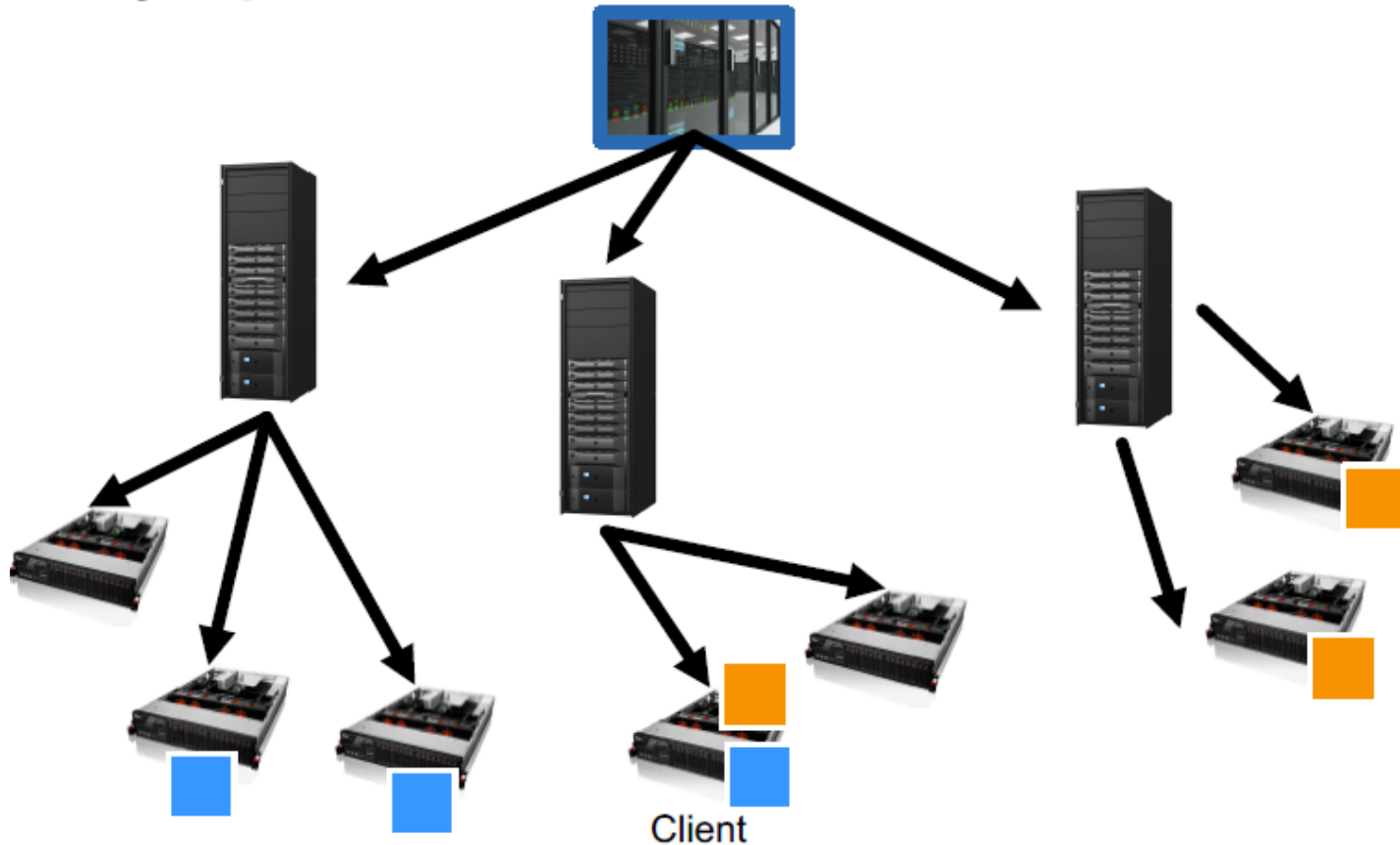
Rack awareness

Replica placement



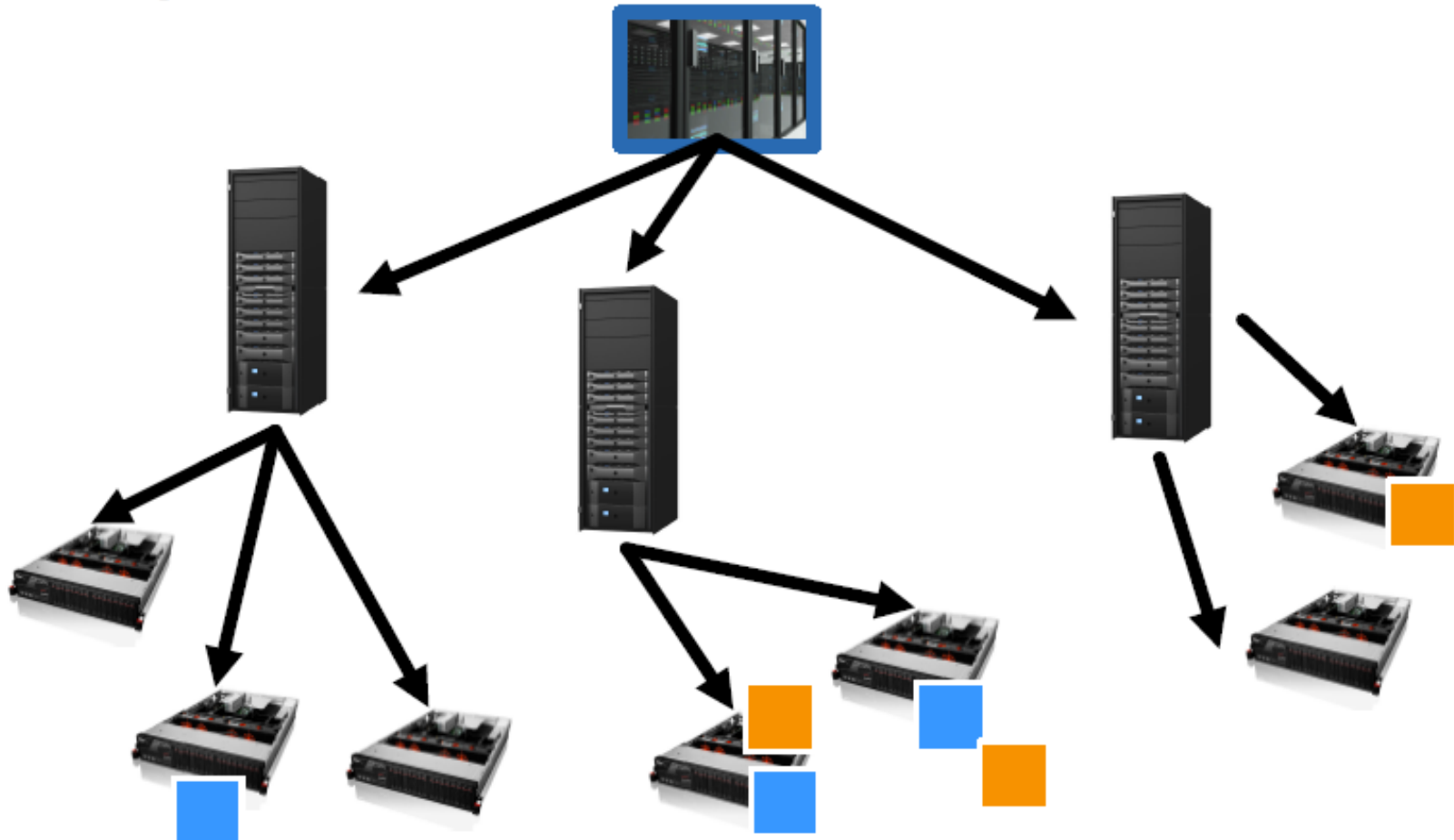
Rack awareness

Why replicas 2+3 on other rack?



Rack awareness

If replicas 1+2 were on same rack...



Block concentration on same rack (2/3)

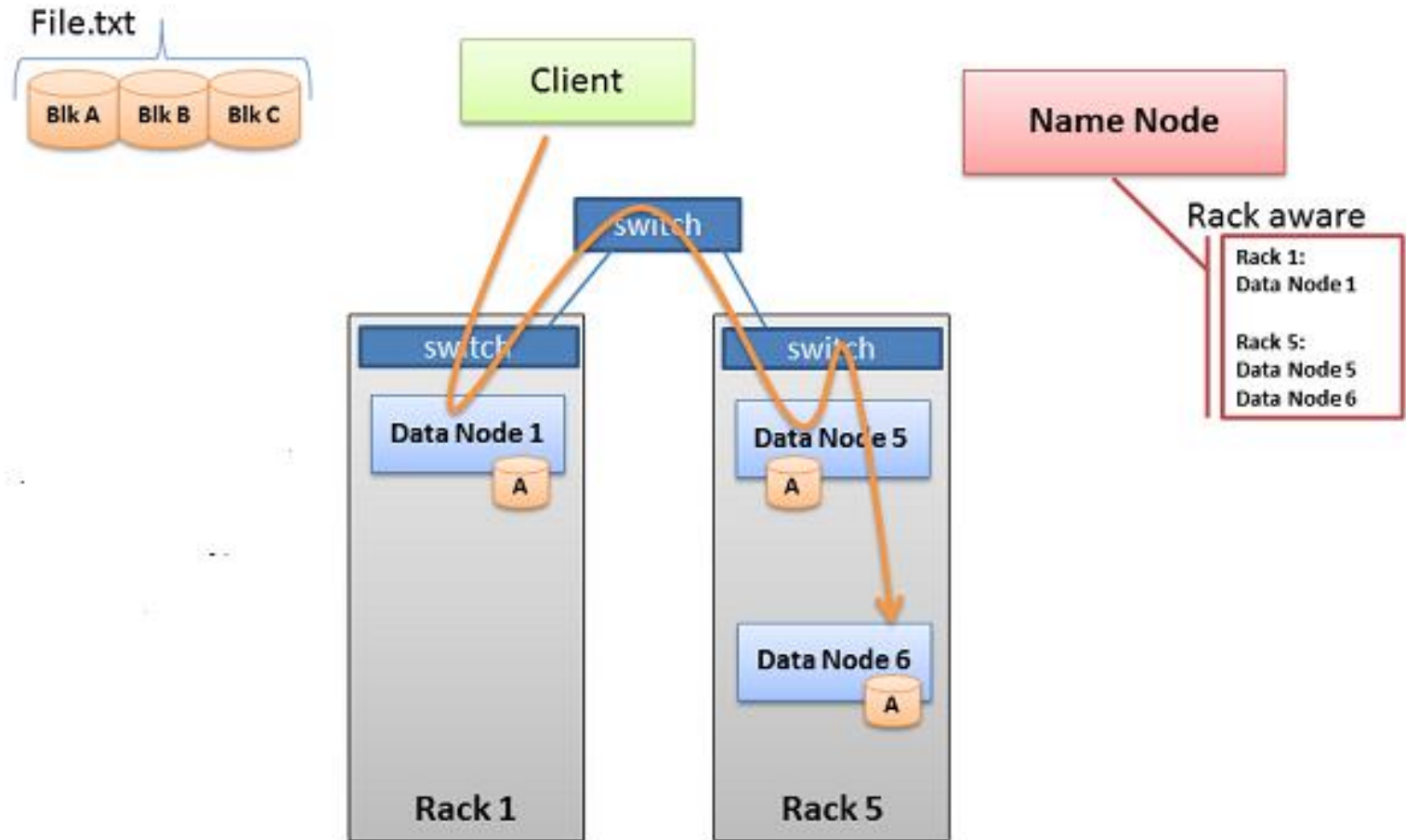
HDFS upis

- Klijent je spreman da loaduje fajl File.txt u klaster (razbija ga u blokove A,B i C) počevši od bloka A.
- Klijent kontaktira Name Node, konsultuje ga, dobija dozvolu od NN i dobija listu od (3) DataNoda za svaki blok koji treba da bude upisan (jedinsvenu listu za svaki blok).
- NN koristi svoje Rack Awareness podatke da bi uticao na odluku koji DataNodovi će se naći na listi za pojedinačni blok
- Ključno pravilo je da za svaki blok podataka, jedna kopija se nalazi u jednom reku, a druge dve kopije u reku različitom od ovog s tim što su te dve kopije u istom reku.
- Sve liste koje dobija Klijent moraju da poštuju ovo pravilo
- Za blok A na listi su DN1, DN5 i DN6.

HDFS upis

- Pre nego što Klijent upiše Blok A fajla File.txt u klaster, on želi da zna da su svi DataNodovi u koje treba da upiše spremni da prime blok.
- Nakon stizanja potvrde, od DN6 do DN5, od DN5 do DN1, i od DN1 do klijenta, Klijent je spreman da upiše blok A.
- Prilikom upisa DN-ovi formiraju pipeline, u redosledu koji minimizira rastojanje od klijenta do poslednjeg DN-a

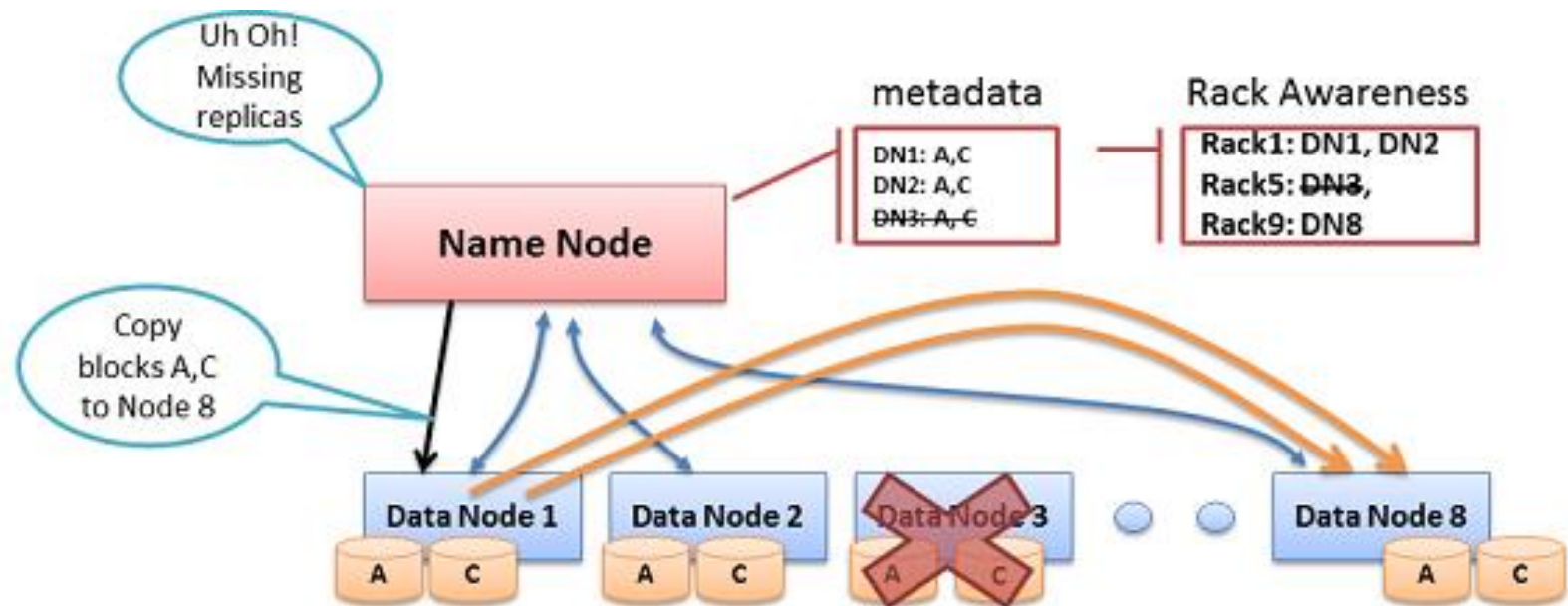
HDFS Upis



Rereplikacija blokova

- Ako NN prestane da prima *heartbeat*-ove od DN on pretpostavlja da je računar “mrtav” i da su podaci na njemu nedostupni.
- Pošto je NN od ovog DN primao izveštaje on zna koji blokovi podataka su bili sačuvani na njemu i može doneti odluku o rereplikaciji blokova na druge DN.
- Pri tome će konsultovati Rack Awareness podatke da bi ispoštovao pravilo da je prva kopija u jednom reku a druge dve kopije u istom u reku, kada odlučuje koji DN će dobiti nove kopije blokova

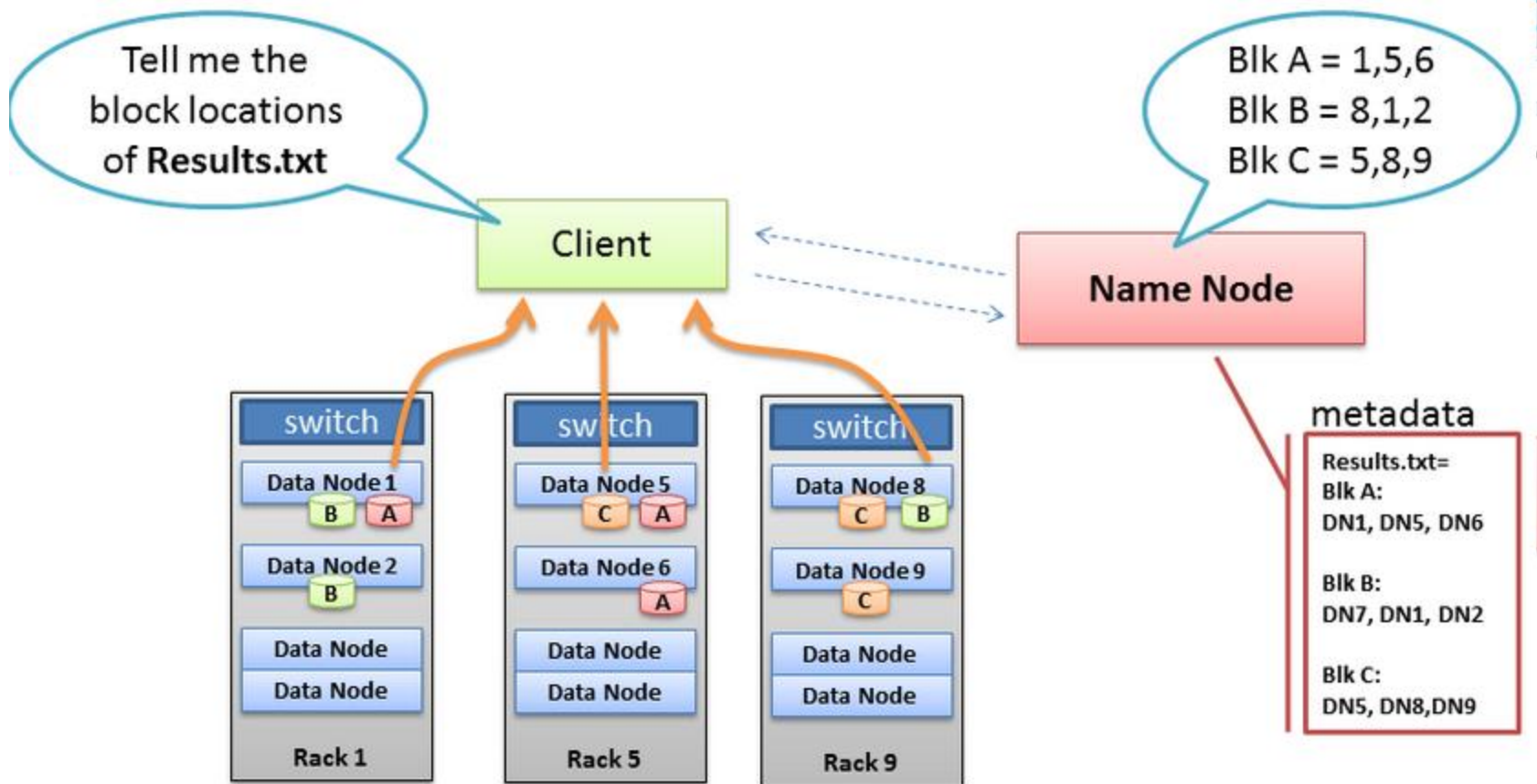
Rereplikacija blokova



Čitanje podataka iz HDFS-a

- Kad Klijent želi da pročita rezultujući fajl iz HDFS, konsultuje NN i pita za lokacije blokova u rezultujućem fajlu
- NN za svaki blok vraća listu DN-ova koji ga sadrže
- Klijent bira za svaki blok prvi DN iz liste (lokacije blokova su sortirane prema udaljenosti od klijenta) tj. najbližu repliku
- Klijent čita blokove sa odgovarajućih DN-a sekvencijalno, jedan po jedan

Čitanje podataka iz HDFS-a



HDFS upis

- Kopiranje ulaznih fajlova sa direktorijuma lokalnog FS **input** u direktorijum **inputdfs** distribuiranog fajl sistema:
- natalija@natalija-VirtualBox:~/Apps/hadoop-1.2.1\$ **bin/hadoop fs -put
../MaxTemperature/input inputdfs**

Web interfejs za HDFS nakon kopiranja

← localhost:50075/browseDirectory.jsp?dir=%2Fuser%2Fnatalija%2Finputdfs&namenodeInfoPort=50070 ☆ ▾

Contents of directory [/user/natalija/inputdfs](#)

Goto :

[Go to parent directory](#)

Name	Type	Size	Replication	Block Size	Modification Time	Permission	Owner	Group
Sample.txt	file	4.59 KB	1	64 MB	2013-12-10 12:04	rw-r--r--	natalija	supergroup

[Go back to DFS home](#)

Kopiranje na lokalni FS

- Kopiranje izlaznih fajlova iz HDFS (outputdfs) u lokalni fajl sistem na folder output2:
- natalija@natalija-VirtualBox:~/Apps/hadoop-1.2.1\$ **bin/hadoop fs -get outputdfs output2**

GFS (Google File System)

GFS vs. HDFS

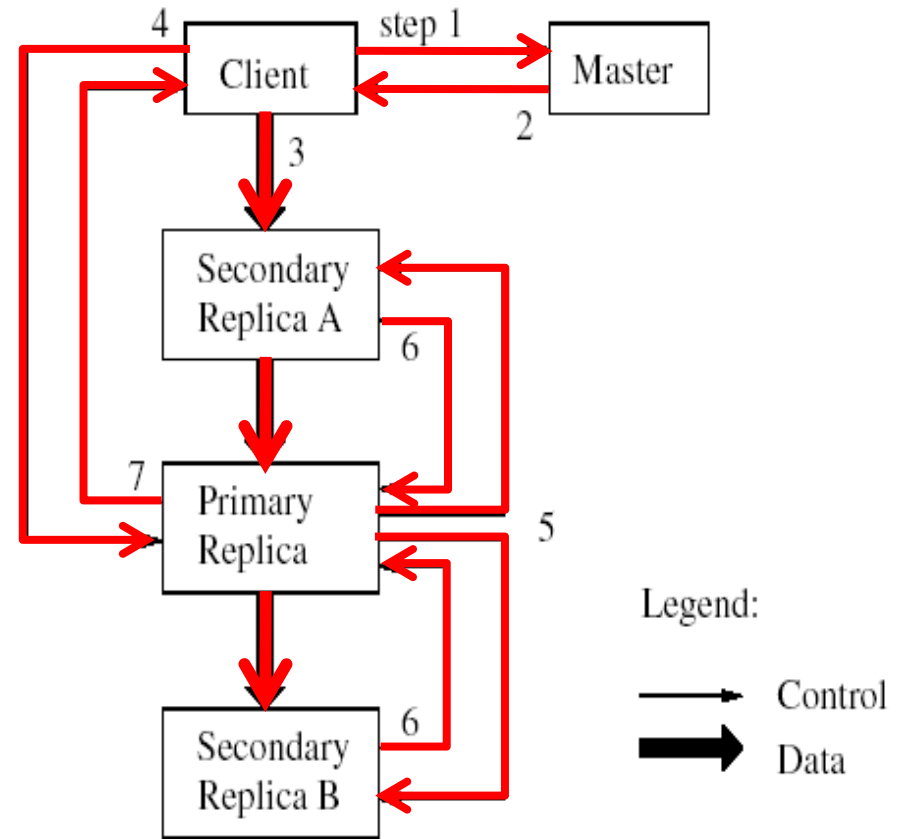
GFS	HDFS
Master	NameNode
chunkserver	DataNode
operation log	journal, edit log
chunk	block
random file writes possible	only append is possible
multiple writer, multiple reader model	single writer, multiple reader model
chunk: 64KB data and 32bit checksum pieces	per HDFS block, two files created on a DataNode: data file & metadata file (checksums, timestamp)
default block size: 64MB	default block size: 128MB

Konzistentnost u GFS-u

- Pošto su chunk-ovi replicirani u GFS-u potrebno je održati konzistentnost replika u slučaju operacija koje modifikuju podatke (mutacija, promena)
- Kada se zahteva mutacija chunk-a, master dodeljuje “zakup” chunk-a (lease) čvoru jedne od replika, koja je sad imenovan kao primarni (primary)
- Ovo je od ključne važnosti kod konkurentnog upisa i ima ulogu u rasterećenju mastera tako što se upravljanje prenosi na chunk server (primary)
- Primary je odgovoran za obezbeđenje serijskog redosleda mutacija za sve konkurentne mutacije istog chunka koje trenutno čekaju
- Zakup koji je dobio Primary omogućava da napravi mutacije nad svojom lokalnom kopijom podataka, ali i da kontroliše i primeni isti redosled mutacija i na ostalim (sekundarnim) kopijama

Mutacije

1. Klijent “pita” master za lokacije replika i da dodeli čvoru na kojoj se nalazi jedna od njih ulogu primary (dodeljuje joj zakup)
2. Master odgovara na upit zahtevanim informacijama
3. Klijent šalje podatke, koji treba da se upišu u chunk, svim čvorovima sa replikama i oni bivaju smešteni u odgovarajući bafer svakog od čvora sa replikom i ne bivaju upisani dok ne dobiju nalog za upis
4. Klijent šalje nalog za upis primary-ju
5. Primary određuje serijski redosled konkurentnih upisa i primenjuje ga i na svoje lokalne podatke i na podatke sekundarnih replika
6. Čvorovi sa sekundarnim replikama odgovaraju Primary da su mutacije uspešno obavljene
7. Primary obaveštava o tome klijenta



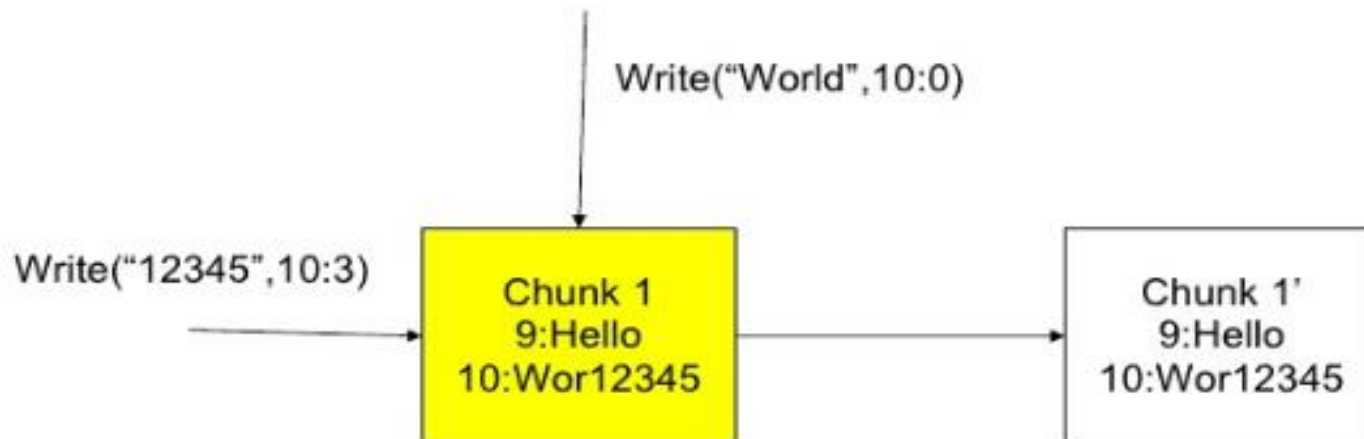
Concurrent Write

- Ako dva klijenta konkurentno vrše upis u isti region fajla nešto od ove dve stvari se može desiti:
 - Preklapajući region može sadržati podatke samo jednog od ta dva upisa
 - Preklapajući region može sadržati miks podataka ova dva upisa

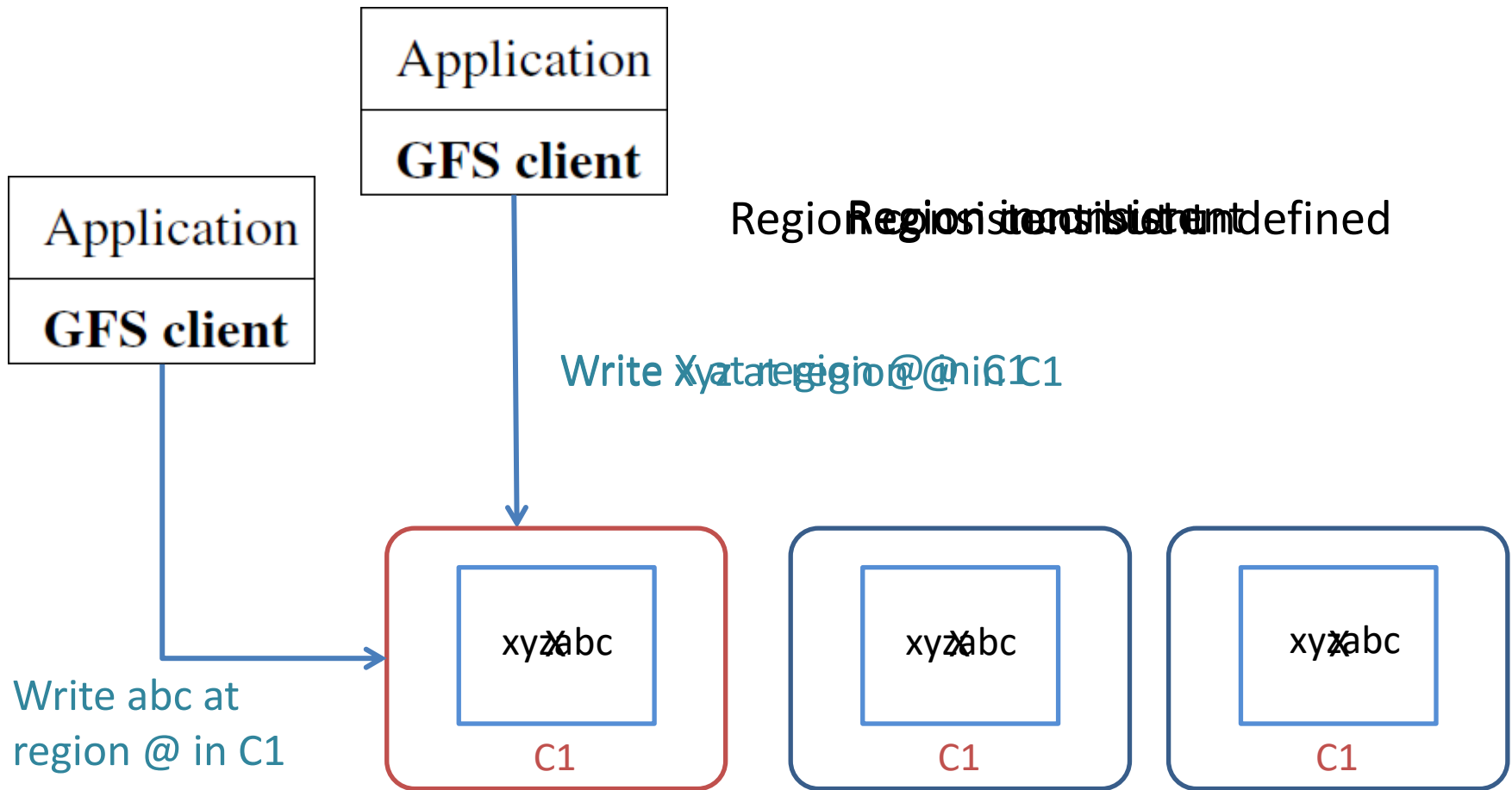
Concurrent Write (primer)

Multiple Writers

Consistent and Undefined



Write/Concurrent Write



HDFS-razlike

- Ne može da se upisuje u fajl koji već postoji u HDFS-u. Moguće je samo dodavati podaci u postojeći fajl
- Ne može više klijenata da upisuje u HDFS fajl konkurentno jer HDFS podržava SingleWriterMultipleReader model
- Klijent koji otvara fajl radi upisa dobija zakup od NN. Ako 2. klijent pokuša istovremeno da upiše traži dozvolu od NN-a, koji proverava da li je izdat zakup za taj fajl i ako jeste 2. klijent će biti odbijen

HDFS-razlike

- Ako jedan klijent upisuje u HDFS fajl, 2. klijent može da čita, ali HDFS ne obezbeđuje da podaci koji su upisivani u fajl budu vidljivi klijentu koji ih čita, dok se fajl ne zatvori